Breaking the Capability Ceiling of LLM Post-Training by Reintroducing Markov States

Yurun Yuan[†] Tengyang Xie[*†]

[†]UW-Madison

March 23, 2026

Abstract

Reinforcement learning (RL) has become a standard paradigm for post-training and aligning Large Language Models (LLMs), yet recent evidence suggests it faces a persistent "capability ceiling": unlike classical RL systems that discover novel strategies, RL for LLMs often acts as a mere refiner of patterns already latent in pre-trained weights. In this work, we identify a fundamental structural bottleneck: while classical RL relies on compact, informative Markov states, current LLM post-training formulations are tethered to an ever-expanding history of actions.

We revisit a classical principle long central to RL yet absent from LLM post-training: explicit Markov states. Theoretically, we provide rigorous guarantees demonstrating that leveraging estimated Markov states can significantly reduce sample complexity. Empirically, we show that introducing Markov states consistently breaks the performance boundaries of standard RL post-training across a suite of complex logic puzzles. Our findings suggest that moving beyond "history-as-state" modeling in favor of structured Markovian representations is essential for unlocking open-ended discovery and genuinely new reasoning capabilities in Generative AI.

1 Introduction

Reinforcement learning (RL) has emerged as the definitive paradigm for post-training and aligning Large Language Models (LLMs), enabling breakthroughs in complex reasoning, mathematical problem-solving, and agentic behavior (Jaech et al., 2024; Guo et al., 2025). By shifting from static supervised fine-tuning to dynamic environment interaction, RL allows models to explore vast solution spaces. In the prevailing RL post-training paradigm, LLMs are formulated as agents where the action space consists of discrete tokens and the state is defined by the concatenation of all preceding actions (Guo et al., 2025).

Despite these successes, growing evidence suggests that RL primarily functions as a mechanism for sharpening search within regions already reachable by the base model, rather than fundamentally expanding its solution space (Yue et al., 2025; Wu et al., 2025a; Shao et al., 2025; Yeo et al., 2025). While some contemporary studies claim that RL can elicit novel capabilities, these gains typically manifest as modest extensions of the pre-training boundary or are contingent upon dense reward shaping or specialized domain-specific designs (Zhang et al., 2025; Sun et al., 2025; Yuan et al., 2025a). Foster et al. (2025) provide theoretical evidence that significant capability expansion is often computationally prohibitive, as the cost of discovery is lower-bounded by either the exponential complexity of the parameter search space or the inherent limitations of the base model's coverage.

This "capability ceiling", however, appears to be a unique artifact of the LLM–RL intersection rather than an inherent limitation of RL itself. In classical RL environments—ranging from robotic manipulation to complex

Email: {yurun_yuan,tx}@cs.wisc.edu
[*]Corresponding author

board games—RL has been serving as a powerful discovery engine rather than a mere capability refiner. For example, systems like AlphaZero (Silver et al., 2017) and MuZero (Schrittwieser et al., 2020) demonstrated the ability to transcend human knowledge, developing novel strategic patterns and superhuman heuristics that were entirely absent from their initial programming or training data. The presence of a performance plateau in RL post-training for LLMs indicates that current formulations may be structurally constrained, necessitating a rethink of foundational assumptions.

A critical distinction emerges when comparing classical RL with its application to LLMs. In classical RL applications, such as robotics or board games like Go, the Markov states are central: a compact representation that encapsulates all information necessary for optimal future decision-making. In contrast, current LLMs operate over a cumulative sequence of previous tokens, relying on an ever-expanding, inherently noisy history rather than a distilled, Markovian representation. Therefore, we argue that this "capability ceiling" is a consequence of the *action-sequence-based* formulation and hypothesize that the reintroduction of the Markov states is the key to unlocking genuinely new reasoning capabilities and improving generalization.

In this paper, we revisit a classical RL principle, explicit Markov state (estimation), and demonstrate its critical importance for LLM post-training. We provide both theoretical foundations and empirical evidence showing that this simple idea yields significant improvements over traditional history-dependent formulations. Our primary contributions are as follows:

- **Breaking the Capability Ceiling:** Through extensive benchmarking on a suite of complex logic puzzles, we show that models with explicit Markov states consistently surpass the performance boundaries of traditional RL post-training, achieving high success rates on tasks where history-dependent models plateau or fail.
- **Robust Generalization:** We demonstrate that Markov models exhibit superior out-of-distribution (OOD) generalization, effectively solving puzzles with higher structural complexity and search depth than those encountered during training.
- **Sample Efficiency Guarantees:** We provide theoretical guarantees demonstrating that Markovian learning achieves significantly lower sample complexity compared to standard action-sequence-based formulations.

Taken together, our findings suggest that the path toward artificial general intelligence and open-ended capability growth may require moving beyond "history-as-state" modeling in favor of Markovian states that better align with the underlying logic of complex reasoning tasks.

2 Preliminaries

2.1 Markov Decision Process, Policies, and Value Functions

RL provides a framework for sequential decision-making problems where an agent interacts with an environment to maximize cumulative rewards. In the context of Markov Decision Processes (MDPs), which provide the theoretical foundation for RL, we consider an episodic finite-horizon framework. Formally, a horizon-H episodic MDP $M = (H, \mathcal{S}, \mathcal{A}, \mathcal{P}, r, \rho)$ consists of a (potentially very large) state space $\mathcal{S}$, an action space $\mathcal{A}$, a probability transition function $\mathcal{P} : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$, a reward function $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$, and an initial state distribution $\rho \in \Delta(\mathcal{S})$. The state space is typically layered such that $\mathcal{S} = \mathcal{S}_1 \cup \mathcal{S}_2 \cup \cdots \cup \mathcal{S}_H$, where $\mathcal{S}_h$ is the set of states reachable at step h. A policy $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ maps states to distributions over actions and induces a distribution over trajectories $\tau = (s_1, a_1, \ldots, s_H, a_H)$ and rewards $(r_1, \ldots, r_H)$, where the initial state is sampled as $s_1 \sim \rho$, and for $h = 1, \ldots, H$: $a_h \sim \pi(s_h)$, $r_h = r(s_h, a_h)$, and $s_{h+1} \sim \mathcal{P}(s_h, a_h)$. We let $\mathbb{E}_{\tau\sim\pi}[\cdot]$ and $\mathbb{P}_{\tau\sim\pi}[\cdot]$ denote expectation and probability under this process, and $\mathbb{E}_\pi[\cdot]$ and $\mathbb{P}_\pi[\cdot]$ for brevity when τ is not explicitly mentioned.

The expected cumulative reward of a policy π is given by $J(\pi) = \mathbb{E}_{\tau\sim\pi}[r(\tau)]$, where $r(\tau) = \sum_{h=1}^{H} r(s_h, a_h)$. The value function and Q-function of π is defined as $V_h^\pi(s) \coloneqq \mathbb{E}_\pi\left[\sum_{\ell=h}^{H} r_\ell(s_\ell, a_\ell) \mid s_h = s\right]$, $Q_h^\pi(s, a) \coloneqq \mathbb{E}_\pi\left[\sum_{\ell=h}^{H} R_\ell(s_\ell, a_\ell) \mid s_h = s, a_h = a\right]$. Additionally, the advantage function A^π represents the relative benefit of taking a specific action a compared to following the policy π on average, defined as: $A_h^\pi(s, a) \coloneqq Q_h^\pi(s, a) -$

$V_h^\pi(s)$. We denote the optimal policy as $\pi^\star$ (i.e., $\pi^\star \in \operatorname{argmax}_\pi J(\pi)$) and its associated value, Q, and advantage functions as $V^\star$, $Q^\star$, and $A^\star$, respectively.

2.2 Reinforcement Learning for Language Models

In the context of language models, the model serves as the policy π, and the input and output of the model maps to the state s and action a respectively. In the single-turn setting where $x \sim \rho$ denotes the input prompt and $y_1, y_2, \ldots, y_H$ denote the output tokens, we can define $s_1 \coloneqq x$ and $s_h \coloneqq (x, y_1, \ldots, y_{h-1})$ for $h > 1$, with $a_h \coloneqq y_h$ for $h = 1, \ldots, H$. In the multi-turn setting, which consists of multiple interaction turns $(x^{(1)}, y_{1:H}^{(1)})$, $(x^{(2)}, y_{1:H}^{(2)})$, and so forth, we can adapt the transition function accordingly. Here, $y_{1:H}^{(i)}$ is a shorthand notation for the sequence of tokens $y_1^{(i)}, y_2^{(i)}, \ldots, y_H^{(i)}$ in the i-th turn. For instance, if a state-action pair (s, a) contains the complete response for one turn (e.g., in a conversation with three or more turns), where $s = (x^{(1)}, y_{1:H}^{(1)}, x^{(2)}, y_{1:H-1}^{(2)})$ and $a = y_H^{(2)}$, the next state would transition to $s' = (x^{(1)}, y_{1:H}^{(1)}, x^{(2)}, y_{1:H}^{(2)}, x^{(3)})$.

In standard RL, the objective is to find a policy π that maximizes the expected cumulative reward $J(\pi) = \mathbb{E}_{\tau\sim\pi}[r(\tau)]$. In many practical applications, particularly in the context of large language models, it is beneficial to incorporate a regularization term that encourages the learned policy to stay close to a reference policy π_{ref}. This leads to the KL-regularized RL objective (Ziebart et al., 2008; Ziebart, 2010; Neu et al., 2017; Ouyang et al., 2022; Xie et al., 2024; Yuan et al., 2025b)

$$J_\beta(\pi) = \mathbb{E}_{\tau\sim\pi}[r(\tau)] - \beta \cdot \mathbb{E}_{\tau\sim\pi}\left[\log \frac{\pi(\tau)}{\pi_{\mathsf{ref}}(\tau)}\right],$$

where $\beta > 0$ is a regularization parameter that controls the strength of the penalty $D_{\mathrm{KL}}(\pi\|\pi_{\mathsf{ref}}) = \mathbb{E}_{\tau\sim\pi}\left[\log \frac{\pi(\tau)}{\pi_{\mathsf{ref}}(\tau)}\right]$, known as the Kullback-Leibler divergence. We use $\pi_\beta^\star \in \operatorname{argmax}_\pi J_\beta(\pi)$ to denote the KL-regularized optimal policy.

Proximal Policy Optimization (PPO; Schulman et al., 2017) and Group Relative Policy Optimization (GRPO; Shao et al., 2024) represent the primary algorithmic frameworks currently utilized for reinforcement learning post-training and alignment. They introduce a clipped surrogate objective to constrain policy updates:

$$\mathcal{J}(\theta) = \mathbb{E}_{(s_h,a_h)\sim\pi_{\theta_{\mathsf{old}}}}\left[\min\left(\frac{\pi_\theta(a_h|s_h)}{\pi_{\theta_{\mathsf{old}}}(a_h|s_h)}\widehat{A}_h(s_h,a_h), \operatorname{clip}\left(\frac{\pi_\theta(a_h|s_h)}{\pi_{\theta_{\mathsf{old}}}(a_h|s_h)}, 1-\varepsilon, 1+\varepsilon\right)\widehat{A}_h(s_h,a_h)\right)\right],$$

where $\widehat{A}_h$ is the advantage estimate, and ε is a hyperparameter. In PPO, the advantage $\widehat{A}_h$ is typically computed using Generalized Advantage Estimation (GAE; Schulman et al., 2015b) to estimate the advantage of the KL regularized reward $r(s_h, a_h) - \beta \log \frac{\pi(a_h|s_h)}{\pi_{\mathsf{ref}}(a_h|s_h)}$. GRPO is a policy-based method that, in practical implementations for LLMs like DeepSeek-R1, samples G responses $o^{(1)}, \ldots, o^{(G)}$ for each prompt x and computes advantages by normalizing rewards within each prompt group. This response-level advantage $\widehat{A}^{(i)}$ is then used to replace the step-wise advantage function $\widehat{A}_h(s_h, a_h)$ in the objective $\mathcal{J}(\theta)$. GRPO objective then accommodates the KL-regularization at the end. GRPO is often considered a simpler alternative to PPO for post-training LLMs. This is partly because PPO typically involves training a separate critic network and incorporates more complex mechanisms for policy updates. In the context of LLMs, the full complexity of PPO might not always be necessary, leading to the adoption of more streamlined policy gradient methods like GRPO.

3 Reintroducing Markov States to LLM Post-Training

3.1 Limits of Current RL for LLMs

Despite the empirical success of RL in improving the reasoning performance of large language models (Guo et al., 2025; Jaech et al., 2024), it remains debated whether RL can induce capabilities that fundamentally exceed those acquired during pre-training. A growing body of evidence suggests that RL primarily reweights or amplifies reasoning patterns already latent in the base model, rather than creating genuinely novel capabilities (Yue et al., 2025; Wu et al., 2025a; Shao et al., 2025; Yeo et al., 2025).

Conversely, reports of emergent capabilities under RL typically rely on restrictive training designs (Yuan et al., 2025a; Zhang et al., 2025; Sun et al., 2025). These mechanisms guide learning toward known solution manifolds, suggesting that the observed gains reflect controlled extrapolation within a limited hypothesis space rather than the discovery of new reasoning trajectories.

Recent work (Foster et al., 2025) also provides theoretical evidence for this fundamental boundary. Let $C_{\mathsf{cov}}(\pi^\star_\beta) := \max_{s,a} \frac{\pi^\star_\beta(y|x)}{\pi_{\mathsf{ref}}(y|x)}$ be the coverage coefficient of the base model π_{ref} w.r.t. the sequence of tokens, which controls the quality of pass@k performance of the base model.

The hope for emergent capabilities under RL is that: if both the statistical and computational complexity of RL were much smaller than $C_{\mathsf{cov}}(\pi^\star_\beta)$ (e.g., Xie et al. 2024 shows that the statistical complexity can be much smaller than $C_{\mathsf{cov}}(\pi^\star_\beta)$ in certain cases), then RL could yield significant gains beyond the base model's pass@k performance. However, Foster et al. (2025) establish the following lower bound on computational complexity: under the KL-regularized RL objective, achieving a near-optimal policy requires the number of sampling oracle calls (and runtime) T_{comp} to be lower-bounded by

$$\Omega\left(\min\left\{C_{\mathsf{cov}}(\pi^\star_\beta), \exp\left(\frac{R_{\mathsf{max}}}{\beta}\right)\right\}\right),$$

even for the linearly-parameterized softmax policy class. R_{max} is an upper bound on the reward $r(\tau)$.

This lower bound reveals a strict "discovery" bottleneck: for RL to find a near-optimal policy efficiently, an algorithm is forced to either **(1)** rely on the base model to already cover the optimal response with small $C_{\mathsf{cov}}(\pi^\star_\beta)$, which in turn implies that the base model's pass@k performance is already reasonable, or **(2)** resort to brute-force exploration over the response space, at a cost that grows exponentially as $e^{R_{\mathsf{max}}/\beta}$. In particular, if the base model's pass@k performance is poor (i.e., $C_{\mathsf{cov}}(\pi^\star_\beta)$ is large), RL is pushed into the exponential-cost regime, making the discovery of truly novel solutions computationally intractable.

Collectively, these findings point to a ceiling of current RL-based post-training paradigms: rather than expanding the model's solution space, RL predominantly sharpens search within regions already accessible to the base model, yielding at most modest extensions beyond its pre-training boundary. This motivates re-examining the foundations of RL for LLMs and casts doubt on whether existing approaches alone can support open-ended capability growth.

3.2 A Didactic Example

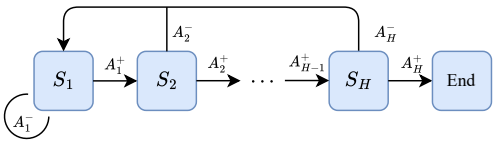


Figure 1: The *Combination Lock* problem with horizon H. At each state S_h, the correct action A_h^+ advances the agent to the next state; the incorrect action A_h^- resets it to the starting position.

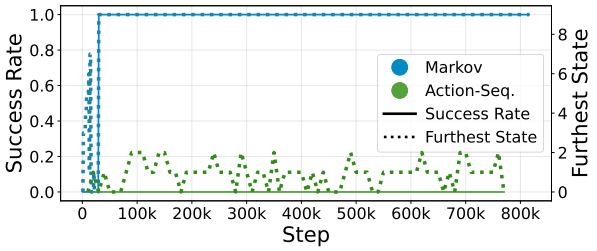


Figure 2: Comparison of Markov agent and action-sequence agent on *Combination Lock* task. We focus on two metrics: the success rate in reaching the final goal state and the furthest state reached before the agent triggers an incorrect action.

We start with an empirical analysis on a didactic task: the *Combination Lock* problem. As illustrated in Figure 1, this environment consists of H linearly ordered states, $(S_1, S_2, \cdots, S_H)$, and two discrete actions. The agent begins at S_1; selecting the correct action A_h^+ advances the agent to the next state, while an incorrect choice A_h^- resets it to the starting position. Each transition incurs a reward of -1, except for

Algorithm 1 Action-Sequence Models (Figure 3(a))

Input: Initial state s_1
for $h = 1, \ldots, H$ **do**
  Sample action $a_h \sim \pi(\cdot \mid s_1, a_{1:h-1})$
  Append a_h to history sequence
end for

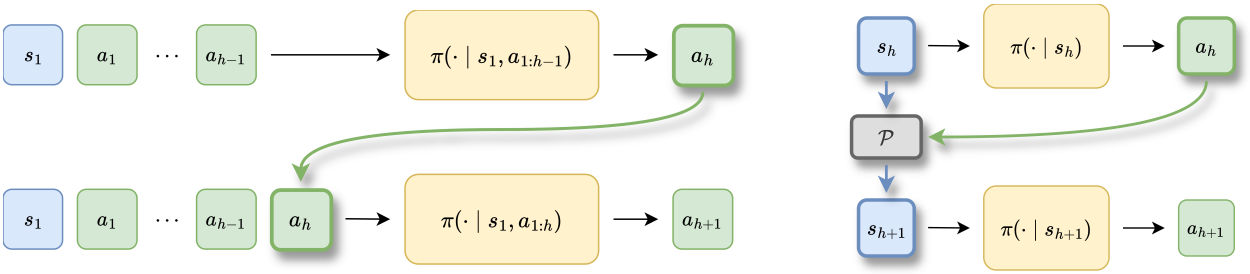
Algorithm 2 Markovian Models (Figure 3(b))

Input: Initial state s_1
for $h = 1, \ldots, H$ **do**
  Sample action $a_h \sim \pi(\cdot \mid s_h)$
  Update state $s_{h+1} \leftarrow \mathcal{P}(s_h, a_h)$
end for

the final goal state, which yields a reward of 0 and terminates the episode. Consequently, the agent must memorize the sequence of H correct actions $(A_1^+, A_2^+, \cdots, A_H^+)$ to reach the goal.

We instantiate this task with a horizon of $H = 10$ and evaluate two multilayer perceptron (MLP)-based agents that approach the problem from distinct modeling perspectives. The first network is Markov-state-based, receiving the encoded representation of the current Markov state s_h as input. The second is action-sequence-based, whose input is the concatenation of all previous actions $(a_1, \ldots, a_{h-1})$. Both agents are trained via Deep Q-Learning (Mnih et al., 2015) to select the action a_h that maximizes cumulative reward. We evaluate performance using two key metrics: the success rate in reaching the final goal state and the furthest state reached before the agent triggers an incorrect action. As shown in Figure 2, the Markov agent successfully memorizes the correct actions and stabilizes within 30k steps, while the action-sequence agent fails to reach the goal even after 800k steps.

The substantial performance gap between the two paradigms is not surprising. For the Markov agent, the input space coincides with the state space and contains only H distinct values while the action-sequence agent operates over an input space consisting of full action histories, suggesting that incorporating Markov states in the inputs is essential for solving this task efficiently.



(a) Action-Sequence Models.

(b) Markov Models.

Figure 3: Comparison of action-sequence-based models and our Markovian Models. For action-sequence models, the new action (a_h) is appended to the existing action history and fed back into the model as the input for the subsequent prediction. For Markov models, the next action a_h is combined with the current state s_h and passed through a state transition function $\mathcal{P}$ to obtain the next state s_{h+1}, which is the input for the subsequent prediction.

3.3 Markov States in LLM Post-Training

In contrast to the evidence in Section 3.2, contemporary LLM post-training practices predominantly adopt an action-sequence-based formulation, where the history of actions is treated as the state, as shown in Algorithm 1. Here, an "action" is broadly defined: it may represent a single token, a semantic step toward the final solution, or an iteration of response-refinement. The pronounced mismatch between prevailing post-training practices and our insights motivates us to rethink about the RL post-training paradigm.

Markov State Estimation. We reintroduce explicit Markov states into the LLM training pipeline. As illustrated in Algorithm 2, the newly generated action a_h, instead of being simply appended to the previous actions, is combined with the current state s_h and passed through a state transition function $\mathcal{P}$. The resulting

state s_{h+1} is then used as the input for the next decision step. By construction, s_{h+1} preserves all information necessary for future actions while discarding irrelevant noise from the interaction history. In practice, the state transition function may be realized by an environment that internally maintains a Markov state, a rule-based mechanism implementing the transition logic, or a learned model that approximates the underlying transition dynamics.

Approach	Sudoku	Sokoban	Futoshiki
Qwen3-4B			
Action-seq.	92.3	2.5	0.2
Markov	97.1	76.1	75.0

Table 1: Comparison of action-sequence models and Markov models on logical reasoning tasks. Across all tasks, Markov models consistently outperform their action-sequence counterparts by a substantial margin.

Empirical Evidence. To empirically validate the advantages of incorporating Markov states, we conduct experiments on a set of synthetic, controllable tasks with well-defined Markov state representations. In particular, we consider a suite of logical reasoning games, including Sudoku, Sokoban, and Futoshiki. For each task, we post-train models using both action-sequence-based and Markov paradigms with the same RL algorithm. We also train a separate state transition estimation model. As summarized in Table 1, Markov models consistently outperform their action-sequence counterparts by a substantial margin, even on tasks where action-sequence models exhibit near-zero accuracy. We defer full experimental details and comprehensive evaluation results to Section 4.

Broader Implications and Applications. The applicability of Markovian language models extends well beyond synthetic benchmarks to a wide range of real-world settings. In many domains, the Markov states are *accessible* during training and our paradigm can easily fit in. To illustrate this broader potential, we outline several representative scenarios: **(1) Coding:** In multi-turn code debugging, the state represents a snapshot of the current codebase together with relevant execution or compiler logs, and evolves through actions such as code edits or test executions. In contrast, an action-sequence-based agent observes only its history of proposed changes, without explicitly reasoning over the resulting code snapshot. (Team, 2025; Hui et al., 2024). **(2) Mathematical reasoning:** The state consists of the set of established lemmas and intermediate results, with each new inference transitioning the system toward a more complete proof (Hubert et al., 2025; Chen et al., 2025). **(3) Iterative response refinement:** The state is restricted to the most recent draft, and the transition function $\mathcal{P}$ overwrites the previous version with the refined output. This design enables the model to reason over the current solution while avoiding redundant noise from its own edit history (Yuan and Xie, 2025). Standard action-sequence-based baselines ignore these Markovian structures, while our paradigm suggests that aligning the agent's representation with the efficient underlying Markov structure enables it to solve complex, long-horizon tasks that are otherwise intractable.

4 Experiments

In this section, we report comprehensive experiments and analyses of Markovian learning and comparison to its counterparts.

Tasks and Datasets. To accurately obtain Markov states in LLM reasoning, we implement three synthetic logical tasks from Reasoning-Gym (Stojanovski et al., 2025): Sudoku, Sokoban, and Futoshiki. These grid-based puzzles challenge a model's capacity for logical analysis, constraint satisfaction, and spatial reasoning. Crucially, the configuration of the board at any given step serves as a fully observable Markov state; every discrete action deterministically updates this configuration to form the subsequent state, yielding an explicit state trajectory for training and analysis.

Approach	In Distribution						Out Of Distribution					
	Sudoku		Sokoban		Futoshiki		Sudoku		Sokoban		Futoshiki	
	Avg	Pass	Avg	Pass	Avg	Pass	Avg	Pass	Avg	Pass	Avg	Pass
Qwen3-4B												
Action-sequence												
+ SFT Warm-up	16.1	98.0	0.2	22.0	0.0	5.0	3.1	64.0	0.0	1.0	0.0	0.0
+ RL post-training	93.5	97.0	2.3	4.0	0.1	7.0	69.2	82.0	0.0	0.0	0.0	0.0
Markov												
+ SFT Warm-up	34.2	100.0	0.4	28.0	0.1	11.0	8.7	86.0	0.0	3.0	0.0	0.0
+ RL post-training	97.1	98.0	76.1	81.0	75.0	85.0	77.8	82.0	31.6	37.0	42.6	53.0
State-action-sequence												
+ SFT Warm-up	8.6	96.0	1.4	61.0	0.3	20.0	1.9	62.0	0.2	15.0	0.0	1.0
+ RL post-training	91.1	96.0	57.4	67.0	44.4	55.0	71.2	82.0	30.2	34.0	16.9	21.0
Qwen2.5-3B-It												
Action-sequence												
+ SFT Warm-up	0.3	23.0	0.5	37.0	6.6	94.0	0.0	0.0	0.0	3.0	0.3	28.0
+ RL post-training	0.0	0.0	1.0	1.0	61.3	84.0	0.0	0.0	0.0	0.0	24.9	56.0
Markov												
+ SFT Warm-up	20.0	99.0	0.2	14.0	8.5	98.0	2.9	75.0	0.0	1.0	0.3	26.0
+ RL post-training	86.0	94.0	89.7	93.0	79.8	96.0	56.4	68.0	66.9	72.0	28.3	67.0
State-action-sequence												
+ SFT Warm-up	22.4	100.0	0.6	41.0	16.6	100.0	2.4	71.0	0.0	1.0	1.1	60.0
+ RL post-training	83.0	90.0	43.6	50.0	67.4	94.0	57.1	69.0	20.4	23.0	25.2	75.0

Table 2: Performance comparison of different approaches. We sample 128 solutions per question and report **Avg@128** and **Pass@128**.

Models and Training Pipelines. We use π_{mkv} to denote the Markov models and $\pi_{\mathsf{act\text{-}seq}}$ to denote the action-sequence models. Appendix C.7 provides illustrative examples of how models operate on these tasks.

Experiments are conducted with Qwen3-4B (Team, 2025) and Qwen2.5-3B-Instruct (Team, 2024), training a separate model for each task. For each model, we first perform a brief task-specific SFT warm-start stage to establish task understanding and output formatting. We then post-train with GRPO (Shao et al., 2024) in an interactive setting, where the agent acts in the true environment with ground-truth transition dynamics $\mathcal{P}^\star$. The agent receives a sparse terminal reward: 1 for solving the instance and 0 otherwise. In addition, we train a state prediction model $\widehat{\mathcal{P}}$ based on Qwen2.5-3B-Instruct via SFT to predict the next state $\widehat{s}_{h+1}$ from the current state and action. At test time, $\widehat{\mathcal{P}}$ replaces the environment $\mathcal{P}^\star$, enabling deployment without environment access.

Implementation Details. We implement our methods and baselines in the rLLM framework (Tan et al., 2025), largely following the recommended hyperparameter settings.

We intentionally require the models to output only the next action without chain-of-thought. This is because the base model is natively trained to solve these puzzles end-to-end; when allowed to reason step by step, it often behaves like an implicit transition model, forecasting future board states inside its reasoning trace (see Appendix C.8 for an illustration). This behavior undermines the goal of decomposing the problem into progressive steps. By constraining the model to outputting only the action, we delegate state-transition computation to an external state prediction model, ensuring that the policy conditions on an explicit next state rather than implicitly inferring it during generation.

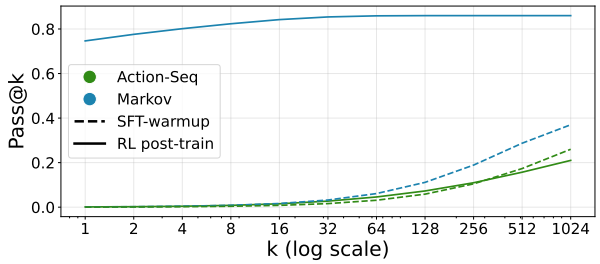


Figure 4: Pass@k accuracy for Qwen3-4B-based models on Futoshiki. While action-sequence models rarely improve SFT Pass@k, Markov models consistently surpass their base models' limits.

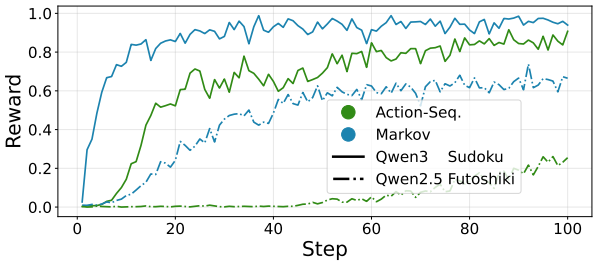


Figure 5: Training reward curves. Markov models reach higher rewards in fewer training steps, providing empirical evidence of lower sample complexity.

4.1 Main Results

We compare the performance of π_{mkv} and $\pi_{\text{act-seq}}$ in Table 2 on two evaluation settings: **(1)** in-distribution (ID) tests, matched to the training set in difficulty and complexity, and **(2)** out-of-distribution (OOD) benchmarks, which are harder than training and typically require more decision steps, thereby probing generalization to greater reasoning depth. For each question, we sample $k = 128$ solutions at temperature 1.0 and report the arithmetic mean of success across all samples, denoted as **Avg@128**, and the probability that at least one of the 128 samples is correct, denoted as **Pass@128**.

Across both settings, π_{mkv} consistently improves one-shot accuracy and Pass@128. The gains are most pronounced on challenging tasks where $\pi_{\text{act-seq}}$ attains near-zero performance—for example, Sokoban and Futoshiki with Qwen3-4B, and Sudoku and Sokoban with Qwen2.5-3B-Instruct. On OOD benchmarks, Markov models generalize strongly, outperforming action-sequence models on every task.

We further examine how the models' capability boundaries shift by analyzing Pass@k accuracy as k scales. We use the results of Qwen3-4B on Futoshiki as an example and show it in Figure 4. We find that action-sequence models fail to extend or even maintain the Pass@k of SFT models, a result consistent with the findings from Yue et al. (2025). In contrast, Markov models break through the capability boundaries of the base models, remarkably extending Pass@k. Evaluation on other tasks is deferred to Appendix C.1.

We also observe faster training-time convergence for Markov models. Using Sudoku with Qwen3-4B and Futoshiki with Qwen2.5-3B-Instruct as representative cases, Figure 5 shows that π_{mkv} reaches higher rewards in fewer training steps, providing empirical evidence of lower sample complexity.

4.2 Factors Behind the Success of Markov Models

To understand why π_{mkv} consistently outperforms $\pi_{\text{act-seq}}$, we decompose their differences into two factors: **(1)** explicit access to Markov states s_h, and **(2)** the use of a Markovian structure. To isolate these effects, we introduce an intermediate baseline, the state-action-sequence model $\pi_{\text{st-act-seq}}$, which conditions on the full history of states and actions. Concretely, at step h, $\pi_{\text{st-act-seq}}$ predicts the next action a_h given $(\{(s_i, a_i)\}_{i=1}^{h-1}, s_h) = (s_1, a_1, s_2, a_2, \cdots, a_{h-1}, s_h)$. Unlike $\pi_{\text{act-seq}}$, $\pi_{\text{st-act-seq}}$ has access to the true state sequence; however, it still predicts actions from the entire trajectory and is therefore not Markovian. As a result, comparing $\pi_{\text{act-seq}}$ to $\pi_{\text{st-act-seq}}$ quantifies the benefit of exposing Markov states, while comparing $\pi_{\text{st-act-seq}}$ to π_{mkv} isolates the additional gains attributable to the Markov property.

In the following, we show that access to Markov states is essential for effective RL learning, and that enforcing a Markov decision structure further improves performance.

Explicit Markov State Conditioning. The comparison of $\pi_{\text{act-seq}}$ and $\pi_{\text{st-act-seq}}$ in Table 2 shows that state-action-sequence models substantially improve logical-reasoning performance, breaking the performance ceiling of action-sequence-based RL training. By conditioning on an explicit state provided by an external transition model, it no longer needs to reconstruct the current board configuration implicitly in its latent

space, offloading the burden of state tracking and prediction. During training, a clear state representation enables rewards to be correctly attributed to state-action pairs, rather than to entangled or partially inferred trajectories. At test time, using an explicit state reduces brittleness caused by noisy internal state estimates and makes the policy's input unambiguous.

Markovian Property. Given access to Markov states, enforcing a Markovian policy structure further improves training efficiency. As shown in Table 2, Markov models consistently outperform state-action-sequence models. This empirical finding is in line with theoretical conclusion that Markov learning has exponentially lower sample complexity.

While it may seem counterintuitive that state-action-sequence models perform worse despite having more input information, this extra context is often a liability rather than an asset. Under the Markov assumption, the current state has encoded all past information relevant to future decision-making. Consequently, the additional historical context provided to state-action-sequence models is redundant and may introduce spurious correlations that complicate learning. To further probe what the state-action-sequence model actually uses after training, we present a controlled ablation in Appendix C.3 showing that state-action-sequence models rely primarily on the current state: removing access to the current state causes performance to collapse, while retaining only the last state preserves a substantial fraction of accuracy.

4.3 The RL Challenge Addressed by Markov Models

The challenges of RL can be largely summarized as the need to balance *exploration* of unknown actions, *credit assignment* under delayed and sparse rewards, and *generalization* from limited experience to large or unseen state spaces (Sutton et al., 1998; Kaelbling et al., 1996). Among these challenges, generalization critically depends on how the agent represents and models the environment state. In this section, we will show that Markov models greatly improve generalization.

To isolate generalization from other factors, we introduce two diagnostic variants: $\pi_{\mathsf{mkv}}^{A^\star}$ and $\pi_{\mathsf{act\text{-}seq}}^{A^\star}$. They follow the same training pipeline as π_{mkv} and $\pi_{\mathsf{act\text{-}seq}}$, except that **(1)** we remove the SFT warm-up stage and directly train on the base models, and **(2)** we replace the estimated response-level advantage $\widehat{A}^{(i)}$ used in GRPO objective (see Section 2.2) with the ground-truth optimal advantage $A^\star$ defined in Section 2.1. Practically, $A^\star$ can be obtained using task-dependent rule-based algorithms, detailed in Appendix D.3. In this way, the credit-assignment difficulty during training is largely removed through providing $A^\star$ and converting the original sparse reward into a dense, per-step learning signal. Exploration is also controlled: the final $\mathsf{softmax}$ layer of the language model functions as Boltzmann exploration mechanism, and we use the same sampling temperature across all models. Under these controls, remaining performance differences primarily reflect differences in generalization.

As reported in Table 3, $\pi_{\mathsf{mkv}}^{A^\star}$ consistently outperforms $\pi_{\mathsf{act\text{-}seq}}^{A^\star}$ across models and tasks. The Markov property enables the model to treat distinct action histories as equivalent whenever they induce the same state, thereby promoting generalization to unseen problems. In contrast, action-sequence-based models must explicitly learn this equivalence; previously unseen action sequences may induce erroneous predictions even when they correspond to the same underlying Markov state. We defer the full results in Appendix C.6.

4.4 Additional Results

Beyond our main results, we provide additional analyses in Appendix C. We report training-time reward curves demonstrating faster convergence of Markov models (Appendix C.2). We also conduct an ablation on the fraction of SFT steps used for policy initialization, showing that Markov models achieve higher rewards with fewer SFT steps (Appendix C.4). Finally, we examine the role of Markov states in purely supervised settings and find that the Markov property plays a limited role under SFT alone (Appendix C.5).

Approach	Sudoku	Sokoban[1]	Futoshiki
Qwen3-4B			
Action-seq. w/ A^*	90.8	18.2	54.8
Markov w/ A^*	97.8	33.0	64.8
Qwen2.5-3B-It			
Action-seq. w/ A^*	28.9	0.2	44.1
Markov w/ A^*	83.4	94.2	54.2

[1] Given the capability constraints of the base models, we use lower-complexity Sokoban tasks in this section.

Table 3: Performance comparison of $\pi^{A^\star}_{\mathsf{mkv}}$, $\pi^{A^\star}_{\mathsf{act\text{-}seq}}$, and $\pi^{A^\star}_{\mathsf{st\text{-}act\text{-}seq}}$ on in-distribution benchmarks.

5 Theoretical Analysis

In this section, we rigorously prove that the introduction of Markov states enables the models to achieving higher performance with lower sample complexity. Throughout this section, we assume the underlying environment is deterministic, governed by a ground-truth transition function $s_{h+1} = \mathcal{P}(s_h, a_h)$.[1] We also assume the reward is bounded $r_h \in [0, 1]$ for all h. Two learning paradigms are formalized as below.

(1) *Action-sequence-based learning*: The policy conditions on the initial state and the action history: $a_h \sim \pi(\cdot \mid s_1, a_{1:h-1})$, where $a_{1:h-1}$ is a shorthand notation for the sequence $a_1, a_2, \cdots, a_{h-1}$.

(2) *Approximate Markovian learning*: The agent has access to an approximate transition function $\widehat{\mathcal{P}} : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$ that estimates the true dynamics $\mathcal{P}^\star$. At each step h, the agent observes the approximate Markov state $\widehat{s}_h \leftarrow \widehat{\mathcal{P}}(\widehat{s}_{h-1}, a_{h-1})$ (when $h = 2$, $\widehat{s}_1 := s_1$), and selects action $a_h \sim \pi(\cdot \mid \widehat{s}_h)$. This setting captures practical scenarios where Markov states are approximately recovered via a learned world model or rule-based state extractor.

We consider a general policy optimization framework applicable to both learning paradigms. Specifically, for each t in $\{1, 2, \ldots, T\}$, the learning algorithm updates the current policy $\pi^{(t)}$ using an approximated advantage $\widehat{A}^{\pi^{(t)}}$, yielding $\pi^{(t+1)}$. This algorithmic framework is sketched in Algorithm 3. We make the following assumption on the resulting policies.

Assumption 1 (Optimization error)**.** *For action-sequence-based models,*

$$\frac{1}{T}\sum_{t=1}^{T}\sum_{h=1}^{H}\mathbb{E}_{\pi^\star}\left[\widehat{A}^{\pi^{(t)}}(s_1, a_{1:h})\right] \le H\varepsilon_{\mathsf{opt}};$$

and for approximate Markovian learning,

$$\frac{1}{T}\sum_{t=1}^{T}\sum_{h=1}^{H}\mathbb{E}_{\pi^\star}\left[\widehat{A}^{\pi^{(t)}}(\widehat{s}_h, a_h)\right] \le H\varepsilon_{\mathsf{opt}}.$$

As detailed in Appendix B.2, when PPO-style algorithms are employed, the optimization error satisfies $\varepsilon_{\mathsf{opt}} = \mathcal{O}\left(\sqrt{1/T}\right)$.

We next bound the error of the estimated advantage function $\widehat{A}^{\pi^{(t)}}(x_h, a_h)$, as stated in Assumption 2.

[1] This assumption naturally holds for many reasoning tasks, including the logical puzzles in our experiments as well as settings like code editing and mathematical proof construction, where actions deterministically update the problem state.

Assumption 2 (Advantage estimation error)**.** *We assume*

$$\mathbb{E}_{\pi^{(t)}}\left[\left(\widehat{A}^{\pi^{(t)}}(x_h, a_h) - A^{\pi^{(t)}}(x_h, a_h)\right)^2\right] \leq \varepsilon_{\mathsf{stat}}^2$$

for all h, where $x_h = (s_1, a_1, \ldots, a_{h-1})$ for the action-sequence-based learning and $x_h = \widehat{s}_h$ for approximate Markovian learning.

We further define the state-action occupancy as $d_h^\pi(s, a) = \mathbb{E}_{\tau\sim\pi}\left[\mathbb{I}\left[s_h = s, a_h = a\right]\right]$ and overload the notation for action-sequence occupancy: $d_h^\pi(s'_1, a'_{1:h}) = \mathbb{E}_{\tau\sim\pi}\left[\mathbb{I}\left[s_1 = s'_1, a_{1:h} = a'_{1:h}\right]\right]$. Under these assumptions, we derive the performance guarantee for action-sequence-based learning in Proposition 1.

Proposition 1 (Performance guarantee of action-sequence-based learning)**.** *For the paradigm of action-sequence-based learning, suppose Assumption 1 and Assumption 2 hold, then we have*

$$J(\pi^\star) - \max_t J(\pi^{(t)}) \leq H\varepsilon_{\mathsf{opt}} + H\sqrt{\max_{t,h}\mathbb{E}_{\pi^{(t)}}\left[\left(\frac{d_h^{\pi^\star}(s_1, a_{1:h})}{d_h^{\pi^{(t)}}(s_1, a_{1:h})}\right)^2\right]}\varepsilon_{\mathsf{stat}}.$$

For approximate Markovian learning, we additionally assume the accuracy of the learned transition model, formalized in Assumption 3, and derive the performance guarantee in Proposition 2.

Assumption 3 (State transition model accuracy)**.** *we assume approximate transition model satisfies*

$$\Pr\left[\widehat{\mathcal{P}}(s, a) \neq \mathcal{P}(s, a)\right] \leq \varepsilon_{\mathcal{P}}$$

for all $s \in \mathcal{S}, a \in \mathcal{A}$.

Proposition 2 (Performance guarantee of approximate Markovian learning)**.** *For the paradigm of approximate Markovian learning, suppose Assumption 1, Assumption 2, and Assumption 3 hold, then we have*

$$J(\pi^\star) - \max_t J(\pi^{(t)}) \leq H\varepsilon_{\mathsf{opt}} + H\sqrt{\max_{t,h}\mathbb{E}_{\pi^{(t)}}\left[\left(\frac{d_h^{\pi^\star}(s_h, a_h)}{d_h^{\pi^{(t)}}(s_h, a_h)}\right)^2\right]}\varepsilon_{\mathsf{stat}} + 2H^3\varepsilon_{\mathcal{P}}.$$

The proof of Proposition 1 and Proposition 2 is deferred to Appendix B.3.

Comparing Proposition 1 and Proposition 2 highlights the key benefit of introducing an approximate transition model. In the action-sequence-based learning (Proposition 1), the density ratio $\frac{d_h^{\pi^\star}(s_1, a_{1:h})}{d_h^{\pi^{(t)}}(s_1, a_{1:h})}$ is defined over full action histories, whose space grows exponentially with the horizon H. Bounding this ratio in the worst case requires the learning policy $\pi^{(t)}$ to cover an exponentially large history space, leading to prohibitive sample complexity (essentially scaling as $|\mathcal{A}|^H$). This is consistent with the computational lower bound of Foster et al. (2025): the coverage coefficient $C_{\mathsf{cov}}(\pi_\beta^\star)$ captures the same fundamental difficulty of covering the optimal policy over an exponentially large response space.

In contrast, the bound for approximate Markovian learning (Proposition 2) effectively depends on the density ratio of the *true* underlying Markov states: $\frac{d_h^{\pi^\star}(s_h, a_h)}{d_h^{\pi^{(t)}}(s_h, a_h)}$, despite the agent operating on approximate states $\widehat{s}_h$. Provided that the true task has a compact state structure (e.g., polynomial in H), this ratio is much easier to bound, implying an exponential reduction in variance. Appendix B.4 illustrates this distinction using *Combination Lock* as an example task.

This improvement comes at the cost of an additive bias term $\mathcal{O}\left(H^3\varepsilon_{\mathcal{P}}\right)$, which is polynomial in the horizon and controlled by the transition model accuracy. This presents a favorable trade-off: *by paying a polynomial price for approximate state transitions, we avoid the potentially exponential coverage cost inherent in action-sequence-based learning.* This is particularly significant in light of the computational barrier established by Foster et al. (2025), where action-sequence RL is forced to pay either the coverage cost $C_{\mathsf{cov}}(\pi_\beta^\star)$ or the exponential exploration cost $e^{R_{\mathsf{max}}/\beta}$—introducing Markov states sidesteps this bottleneck by reducing the effective coverage requirement from action histories to the compact state space.

6 Conclusion

In this work, we reintroduce Markov states into LLM post-training and demonstrate their potential to overcome the performance plateau of contemporary post-training paradigms. We hope this perspective motivates future work to incorporate Markovian structure into real-world, complex tasks, paving the way toward more scalable and open-ended capability growth in generative AI.

Acknowledgements

We acknowledge support of the DARPA AIQ Award. This work used the DeltaAI system at the National Center for Supercomputing Applications [award OAC 2320345] through allocation CIS251426 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program, which is supported by U.S. National Science Foundation grants #2138259, #2138286, #2138307, #2137603, and #2138296.

References

Milad Aghajohari, Kamran Chitsaz, Amirhossein Kazemnejad, Sarath Chandar, Alessandro Sordoni, Aaron Courville, and Siva Reddy. The markovian thinker: Architecture-agnostic linear scaling of reasoning. *arXiv preprint arXiv:2510.06557*, 2025.

Qi Cai, Zhuoran Yang, Chi Jin, and Zhaoran Wang. Provably efficient exploration in policy optimization. In *International Conference on Machine Learning*, pages 1283–1294. PMLR, 2020.

Nicolo Cesa-Bianchi and Gábor Lugosi. *Prediction, learning, and games*. Cambridge university press, 2006.

Jiangjie Chen, Wenxiang Chen, Jiacheng Du, Jinyi Hu, Zhicheng Jiang, Allan Jie, Xiaoran Jin, Xing Jin, Chenggang Li, Wenlei Shi, et al. Seed-prover 1.5: Mastering undergraduate-level theorem proving via learning from experience. *arXiv preprint arXiv:2512.17260*, 2025.

Ching-An Cheng, Tengyang Xie, Nan Jiang, and Alekh Agarwal. Adversarially trained actor critic for offline reinforcement learning. In *International Conference on Machine Learning*, pages 3852–3878. PMLR, 2022.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

Eyal Even-Dar, Sham M Kakade, and Yishay Mansour. Online markov decision processes. *Mathematics of Operations Research*, 34(3):726–736, 2009.

Dylan J Foster, Zakaria Mhammedi, and Dhruv Rohatgi. Is a good foundation necessary for efficient reinforcement learning? the computational role of the base model in exploration. *arXiv preprint arXiv:2503.07453*, 2025.

Matthieu Geist, Bruno Scherrer, and Olivier Pietquin. A theory of regularized markov decision processes. In *International conference on machine learning*, pages 2160–2169. PMLR, 2019.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Elad Hazan et al. Introduction to online convex optimization. *Foundations and Trends® in Optimization*, 2 (3-4):157–325, 2016.

Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklós Z Horváth, Goran Žužić, Eric Wieser, Aja Huang, Julian Schrittwieser, Yannick Schroecker, Hussain Masoom, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, pages 1–3, 2025.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.

Xiang Ji, Sanjeev Kulkarni, Mengdi Wang, and Tengyang Xie. Self-play with adversarial critic: Provable and scalable offline alignment for language models. *arXiv preprint arXiv:2406.04274*, 2024.

Leslie Pack Kaelbling, Michael L Littman, and Andrew W Moore. Reinforcement learning: A survey. *Journal of artificial intelligence research*, 4:237–285, 1996.

Sham M Kakade. A natural policy gradient. *Advances in neural information processing systems*, 14, 2001.

Sham M. Kakade and John Langford. Approximately optimal approximate reinforcement learning. In *International Conference on Machine Learning*, 2002. URL https://api.semanticscholar.org/CorpusID:31442909.

Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. Compressing context to enhance inference efficiency of large language models. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 6342–6353, 2023.

Zongqian Li, Yinhong Liu, Yixuan Su, and Nigel Collier. Prompt compression for large language models: A survey. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7182–7195, 2025.

Miao Lu, Weiwei Sun, Weihua Du, Zhan Ling, Xuesong Yao, Kang Liu, and Jiecao Chen. Scaling llm multi-turn rl with end-to-end summarization-based context management. *arXiv preprint arXiv:2510.06727*, 2025.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.

Gergely Neu, Anders Jonsson, and Vicenç Gómez. A unified view of entropy-regularized markov decision processes. *arXiv preprint arXiv:1705.07798*, 2017.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Mihir Prabhudesai, Lili Chen, Alex Ippoliti, Katerina Fragkiadaki, Hao Liu, and Deepak Pathak. Maximizing confidence alone improves reasoning. *arXiv preprint arXiv:2505.22660*, 2025.

Zile Qiao, Guoxin Chen, Xuanzhong Chen, Donglei Yu, Wenbiao Yin, Xinyu Wang, Zhen Zhang, Baixuan Li, Huifeng Yin, Kuan Li, et al. Webresearcher: Unleashing unbounded reasoning capability in long-horizon agents. *arXiv preprint arXiv:2509.13309*, 2025.

Soumya Rani Samineni, Durgesh Kalwar, Karthik Valmeekam, Kaya Stechly, and Subbarao Kambhampati. Rl in name only? analyzing the structural assumptions in rl post-training for llms. *arXiv preprint arXiv:2505.13697*, 2025.

Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, et al. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020.

John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International conference on machine learning*, pages 1889–1897. PMLR, 2015a.

John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015b.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Rulin Shao, Shuyue Stella Li, Rui Xin, Scott Geng, Yiping Wang, Sewoong Oh, Simon Shaolei Du, Nathan Lambert, Sewon Min, Ranjay Krishna, et al. Spurious rewards: Rethinking training signals in rlvr. *arXiv preprint arXiv:2506.10947*, 2025.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv: 2409.19256*, 2024.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.

Zafir Stojanovski, Oliver Stanley, Joe Sharratt, Richard Jones, Abdulhakeem Adefioye, Jean Kaddour, and Andreas Köpf. Reasoning gym: Reasoning environments for reinforcement learning with verifiable rewards, 2025. URL https://arxiv.org/abs/2505.24760.

Yiyou Sun, Yuhan Cao, Pohao Huang, Haoyue Bai, Hannaneh Hajishirzi, Nouha Dziri, and Dawn Song. Rl grokking recipe: How does rl unlock and transfer new algorithms in llms? *arXiv preprint arXiv:2509.21016*, 2025.

Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*. MIT press Cambridge, 1998.

Sijun Tan, Michael Luo, Colin Cai, Tarun Venkat, Kyle Montgomery, Aaron Hao, Tianhao Wu, Arnav Balyan, Manan Roongta, Chenguang Wang, Li Erran Li, Raluca Ada Popa, and Ion Stoica. rllm: A framework for post-training language agents. https://pretty-radio-b75.notion.site/rLLM-A-Framework-for-Post-Training-Language-Agents-21b81902c146819db63cd98a54ba5f31, 2025. Notion Blog.

Qwen Team. Qwen2.5: A party of foundation models, September 2024. URL https://qwenlm.github.io/blog/qwen2.5/.

Qwen Team. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

Fengwei Teng, Quan Shi, Zhaoyang Yu, Jiayi Zhang, Yuyu Luo, Chenglin Wu, and Zhijiang Guo. Atom of thoughts for markov llm test-time scaling. *arXiv preprint arXiv:2502.12018*, 2025.

Scott Viteri, Max Lamparth, Peter Chatain, and Clark Barrett. Markovian transformers for informative language modeling. *arXiv preprint arXiv:2404.18988*, 2024.

Fang Wu, Weihao Xuan, Ximing Lu, Mingjie Liu, Yi Dong, Zaid Harchaoui, and Yejin Choi. The invisible leash: Why rlvr may or may not escape its origin. *arXiv preprint arXiv:2507.14843*, 2025a.

Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Xinmiao Yu, Dingchu Zhang, Yong Jiang, et al. Resum: Unlocking long-horizon search intelligence via context summarization. *arXiv preprint arXiv:2509.13313*, 2025b.

Tengyang Xie, Ching-An Cheng, Nan Jiang, Paul Mineiro, and Alekh Agarwal. Bellman-consistent pessimism for offline reinforcement learning. *Advances in neural information processing systems*, 34:6683–6694, 2021.

Tengyang Xie, Dylan J Foster, Akshay Krishnamurthy, Corby Rosset, Ahmed Awadallah, and Alexander Rakhlin. Exploratory preference optimization: Harnessing implicit q*-approximation for sample-efficient rlhf. *arXiv preprint arXiv:2405.21046*, 2024.

Amy Xin, Jinxin Liu, Zijun Yao, Zhicheng Lee, Shulin Cao, Lei Hou, and Juanzi Li. Atomr: Atomic operator-empowered large language models for heterogeneous knowledge reasoning. *arXiv preprint arXiv:2411.16495*, 2024.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025.

Edward Yeo, Yuxuan Tong, Morry Niu, Graham Neubig, and Xiang Yue. Demystifying long chain-of-thought reasoning in llms. *arXiv preprint arXiv:2502.03373*, 2025.

Lifan Yuan, Weize Chen, Yuchen Zhang, Ganqu Cui, Hanbin Wang, Ziming You, Ning Ding, Zhiyuan Liu, Maosong Sun, and Hao Peng. From $f(x)$ and $g(x)$ to $f(g(x))$: Llms learn new skills in rl by composing old ones. *arXiv preprint arXiv:2509.25123*, 2025a.

Yurun Yuan and Tengyang Xie. Reinforce LLM reasoning through multi-agent reflection. In *Forty-second International Conference on Machine Learning*, 2025. URL https://openreview.net/forum?id=6k3oFS3Lbl.

Yurun Yuan, Fan Chen, Zeyu Jia, Alexander Rakhlin, and Tengyang Xie. Trajectory bellman residual minimization: A simple value-based method for llm reasoning. *arXiv preprint arXiv:2505.15311*, 2025b.

Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025.

Charlie Zhang, Graham Neubig, and Xiang Yue. On the interplay of pre-training, mid-training, and rl on reasoning language models. *arXiv preprint arXiv:2512.07783*, 2025.

Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19724–19731, 2024.

Pei Zhou, Jay Pujara, Xiang Ren, Xinyun Chen, Heng-Tze Cheng, Quoc V Le, Ed Chi, Denny Zhou, Swaroop Mishra, and Huaixiu Steven Zheng. Self-discover: Large language models self-compose reasoning structures. *Advances in Neural Information Processing Systems*, 37:126032–126058, 2024.

Brian D Ziebart. *Modeling purposeful adaptive behavior with the principle of maximum causal entropy*. Carnegie Mellon University, 2010.

Brian D Ziebart, Andrew L Maas, J Andrew Bagnell, and Anind K Dey. Maximum entropy inverse reinforcement learning. *AAAI Conference on Artificial Intelligence*, 8:1433–1438, 2008.

Yuxin Zuo, Kaiyan Zhang, Li Sheng, Shang Qu, Ganqu Cui, Xuekai Zhu, Haozhan Li, Yuchen Zhang, Xinwei Long, Ermo Hua, et al. Ttrl: Test-time reinforcement learning. *arXiv preprint arXiv:2504.16084*, 2025.

Appendix

A Related Work

Debate on RL Post-Training. An active debate has emerged regarding whether RL can endow models with reasoning capabilities that fundamentally exceed those acquired during pre-training. A growing body of work suggests that RL primarily refines, reweights, or selectively amplifies reasoning patterns already latent in the base model, rather than inducing genuinely novel capabilities (Shao et al., 2024; Yue et al., 2025; Wu et al., 2025a; Shao et al., 2025; Yeo et al., 2025). This view is further reinforced by recent self-improvement methods that eliminate environment interaction or external reward signals altogether, yet achieve comparable gains (Prabhudesai et al., 2025; Zuo et al., 2025), indicating that improvement often arises from internal redistribution of probability mass rather than exploratory discovery.

In contrast, works reporting emergent capabilities under RL typically depend on restrictive training designs, such as prerequisite domain knowledge (Yuan et al., 2025a), carefully curated task difficulty (Zhang et al., 2025), or explicitly designed warm-up phases and partial reward shaping (Sun et al., 2025). These mechanisms substantially constrain the optimization landscape and guide learning toward known solution manifolds, suggesting that the observed gains reflect controlled extrapolation within a narrow hypothesis space rather than the discovery of fundamentally new reasoning trajectories.

Furthermore, Samineni et al. (2025) offers a critical reassessment of the prevailing "history-as-state" formulation in RL post-training. Combining theoretical arguments with empirical evidence, they show that mainstream RL post-training methods are effectively equivalent to outcome-conditioned supervised learning, implying that—under this formulation—RL has not meaningfully exceeded the representational or optimization capabilities of supervised learning.

Context Management. A growing body of work addresses context-length explosion by reducing the amount of information carried forward, either through compression (Li et al., 2023, 2025) or by external memory mechanisms (Packer et al., 2023; Chhikara et al., 2025; Xu et al., 2025; Zhong et al., 2024). These methods typically discard irrelevant details and condense salient information into compact summaries to operate over long contexts. In agentic AI, related context management techniques aim to prevent unbounded growth of interaction histories between an agent and its environment, thereby alleviating context limits and improving the stability of long-horizon training (Wu et al., 2025b; Lu et al., 2025).

Replacing the full history with a concise summary breaks the strict "history-as-state" formulation. However, it only *appears* to yield a Markovian state—compression alone does not ensure the Markov property. A Markov state must be a sufficient statistic for optimal future control: any two histories mapped to the same state should induce identical conditional transition and reward distributions. Existing summarization-based approaches do not enforce this sufficiency, leaving the policy to implicitly learn equivalence across exponentially many trajectories. Moreover, without internalizing the environment's transition dynamics, a summarization model may not reliably produce valid Markov states from action sequences.

Markov LLM Reasoning. A few works mitigate reliance on historical information by decomposing problems into atomic reasoning steps and exploring Markovian reasoning processes (Xin et al., 2024; Teng et al., 2025; Zhou et al., 2024). As an example, Atom of Thoughts (AOT) (Teng et al., 2025) proposes a test-time reasoning framework that iteratively transforms a problem into a sequence of answer-equivalent but progressively simpler subquestions. At each iteration, the current question is decomposed into a temporary dependency structure and then contracted into a new subquestion, which serves as a Markov state for the next step, eliminating reliance on historical reasoning traces. These work focuses on improving test-time abilities through planning and decomposition.

Additionally, Markovian Thinker (Aghajohari et al., 2025) structures LLM reasoning into fixed-size chunks, limiting the length of each reasoning step. They find that during RL training the policy learns to write a textual state near the end of each chunk sufficient for seamless continuation of reasoning after reset. Although their work suggests a specific way to obtain fixed-size textual state, our work systematically showcases and

analyzes the benefits of introducing Markov states without additional assumptions on the method of state generation.

Furthermore, Viteri et al. (2024) propose a framework that aims at mitigating unfaithful chain-of-thought reasoning. It consists of a Chain-of-Thought (CoT) generator and a downstream policy that produces the final answer conditioned solely on the generated CoT, thereby treating the CoT as a load-bearing Markov state.

Finally, WebResearcher (Qiao et al., 2025) can be viewed as applying explicit Markov state estimation to DeepResearch, training a single model to predict both the next state and the next action. However, its primary contribution is a new training algorithm for DeepResearch agents, while our work presents a focused study on the impact of Markov property.

B Theoretical Analysis of Markovian Efficiency

In this section, we provide theoretical analysis regarding the benefit of Markov states additional to Section 5.

B.1 General Policy Optimization Protocol

Algorithm 3 General Policy Optimization Protocol

1: **Initialize:** Initial policy $\pi^{(1)}$ and total iterations T.
2: **for** $t = 1$ to T **do**
3: Estimate approximate advantage function $\widehat{A}^{\pi^{(t)}}$ based on current policy $\pi^{(t)}$.
4: Update policy to obtain $\pi^{(t+1)}$ by optimizing objective involving $\widehat{A}^{\pi^{(t)}}$:

$$\pi^{(t+1)} \leftarrow \mathsf{Optimize}\left(\pi^{(t)}, \widehat{A}^{\pi^{(t)}}\right)$$

5: **end for**
6: **Return:** Sequence of policies $\pi^{(1)}, \pi^{(2)}, \ldots, \pi^{(T+1)}$

This policy optimization protocol encompasses many popular algorithms. For PPO, the function $\mathsf{Optimize}$ is defined as

$$\mathsf{Optimize}\left(\pi^{(t)}, \widehat{A}^{\pi^{(t)}}\right) = \operatorname*{argmax}_{\pi} \mathbb{E}_{\pi^{(t)}}\left[\min\left(\frac{\pi(a_h|s_h)}{\pi^{(t)}(a_h|s_h)}\widehat{A}_h^{\pi^{(t)}}(s_h,a_h), \mathrm{clip}\left(\frac{\pi(a_h|s_h)}{\pi^{(t)}(a_h|s_h)}, 1-\varepsilon, 1+\varepsilon\right)\widehat{A}_h^{\pi^{(t)}}(s_h,a_h)\right)\right].$$

For GRPO, $\mathsf{Optimize}$ is

$$\mathsf{Optimize}\left(\pi^{(t)}, \widehat{A}^{\pi^{(t)}}\right) = \operatorname*{argmax}_{\pi} \mathbb{E}_{x\sim\rho,\{o^{(i)}\}_{i=1}^{G}\sim\pi^{(t)}(\cdot|x)}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{h=1}^{|o_i|}\left\{\min\left[\frac{\pi(a_h^{(i)} \mid s_h^{(i)})}{\pi^{(t)}(a_h^{(i)} \mid s_h^{(i)})}\widehat{A}_{(i)}^{\pi^{(t)}}, \mathrm{clip}\left(\frac{\pi(a_h^{(i)} \mid s_h^{(i)})}{\pi^{(t)}(a_h^{(i)} \mid s_h^{(i)})}, 1-\epsilon, 1+\epsilon\right)\widehat{A}_{(i)}^{\pi^{(t)}}\right] - \beta D_{\mathrm{KL}}(\pi\|\pi_{\mathsf{ref}})\right\}\right],$$

B.2 Discussion of Assumption 1

To argue the validity of Assumption 1, we consider the connection between the natural policy gradient (Kakade, 2001) and KL-regularized policy optimization in proximal policy optimization (Schulman et al., 2015a, 2017; Cai et al., 2020). For ease of presentation, we define x_h to be the essential state at time step h ($x_h = (s_1, a_1, \ldots, a_{h-1})$ for the action-sequence-based learning and $x_h = \widehat{s}_h$ for approximate Markovian learning). With this notation, we consider the following RL objectives

$$\pi^{(t+1)} \leftarrow \operatorname*{argmax}_{\pi} \sum_{h=1}^{H} \mathbb{E}_{\pi^{(t)}}\left[\frac{\pi(a_h \mid x_h)}{\pi^{(t)}(a_h \mid x_h)}\widehat{A}^{\pi^{(t)}}(x_h,a_h) - \beta D_{\mathrm{KL}}(\pi(\cdot \mid x_h)\|\pi^{(t)}(\cdot \mid x_h))\right].$$

This objective can be viewed as the approximation of popular RL algorithms for LLMs (we omit clipping and multiple rollouts below, since our focus is on convergence behavior). We can rewrite that objective as

$$\pi^{(t+1)} \leftarrow \operatorname*{argmax}_{\pi} \sum_{h=1}^{H} \mathbb{E}_{x_h \sim \pi^{(t)}} \left[\widehat{A}^{\pi^{(t)}}(x_h, \pi) - \beta D_{\mathrm{KL}}(\pi(\cdot \mid x_h) \| \pi^{(t)}(\cdot \mid x_h)) \right],$$

where $A(x, \pi) \coloneqq \mathbb{E}_{a \sim \pi}[A(x, a)]$. Then, we can easily verify that one global optimum of this objective is

$$\pi^{(t+1)}(\cdot \mid x_h) \propto \pi^{(t)}(\cdot \mid x_h) \cdot \exp\left(\frac{1}{\beta} \widehat{A}^{\pi^{(t)}}(x_h, \cdot) \right),$$

which implies Assumption 1 with $\varepsilon_{\mathsf{opt}} = O(\sqrt{1/T})$. There is a rich literature studied or used this argument from online learning (Cesa-Bianchi and Lugosi, 2006; Even-Dar et al., 2009; Hazan et al., 2016; Neu et al., 2017; Geist et al., 2019) to recent RL advances (Cai et al., 2020; Xie et al., 2021; Cheng et al., 2022; Ji et al., 2024).

B.3 Proof of Proposition 1 and Proposition 2

We first introduce an episodic version of performance difference lemma (Kakade and Langford, 2002).

Lemma 3 (Performance difference lemma)**.** *For any policy π' and π, we have*

$$J(\pi') - J(\pi) = \sum_{h=1}^{H} \mathbb{E}_{(s_h, a_h) \sim d_h^{\pi'}} \left[A_h^{\pi}(s_h, a_h) \right]$$

We provide the proof for these performance guarantees below.

Proof of Proposition 1. For each $\pi^{(t)}$, we have

$$\begin{aligned}
& J(\pi^\star) - \frac{1}{T} \sum_{t=1}^{T} J(\pi^{(t)}) \\
= & \frac{1}{T} \sum_{t=1}^{T} \sum_{h=1}^{H} \mathbb{E}_{\pi^\star} \left[A^{\pi^{(t)}}(s_1, a_1, \ldots, a_h) \right] \\
= & \underbrace{\frac{1}{T} \sum_{t=1}^{T} \sum_{h=1}^{H} \mathbb{E}_{\pi^\star} \left[\widehat{A}^{\pi^{(t)}}(s_1, a_1, \ldots, a_h) \right]}_{\text{(I)}} + \underbrace{\frac{1}{T} \sum_{t=1}^{T} \sum_{h=1}^{H} \mathbb{E}_{\pi^\star} \left[A^{\pi^{(t)}}(s_1, a_1, \ldots, a_h) - \widehat{A}^{\pi^{(t)}}(s_1, a_1, \ldots, a_h) \right]}_{\text{(II)}}.
\end{aligned}$$

For a fixed h, let $x_h = (s_1, a_1, \ldots, a_{h-1})$ and $\tau_h = (s_1, a_{1:h}) = (x_h, a_h)$. The first equation is a direct application of Lemma 3, treating $(s_1, a_1, \ldots, a_{h-1})$ as the essential state x_h.

Term (I) can be bounded as $\leq H\varepsilon_{\mathsf{opt}}$ by Assumption 1. We now bound the term (II). We have:

$$\begin{aligned}
\mathbb{E}_{\pi^\star} \left[A^{\pi^{(t)}}(\tau_h) - \widehat{A}^{\pi^{(t)}}(\tau_h) \right] &= \sum_{\tau_h \in \mathcal{S}_1 \times \mathcal{A}^h} d_h^{\pi^\star}(\tau_h) (A^{\pi^{(t)}}(\tau_h) - \widehat{A}^{\pi^{(t)}}(\tau_h)) \\
&= \sum_{\tau_h \in \mathcal{S}_1 \times \mathcal{A}^h} d_h^{\pi^{(t)}}(\tau_h) \frac{d_h^{\pi^\star}(\tau_h)}{d_h^{\pi^{(t)}}(\tau_h)} (A^{\pi^{(t)}}(\tau_h) - \widehat{A}^{\pi^{(t)}}(\tau_h)) \\
&\leq \sqrt{\mathbb{E}_{\pi^{(t)}} \left[\left(\frac{d_h^{\pi^\star}(\tau_h)}{d_h^{\pi^{(t)}}(\tau_h)} \right)^2 \right]} \sqrt{\mathbb{E}_{\pi^{(t)}} [(A^{\pi^{(t)}}(\tau_h) - \widehat{A}^{\pi^{(t)}}(\tau_h))^2]}
\end{aligned}$$

$$\leq \sqrt{\mathbb{E}_{\pi^{(t)}}\left[\left(\frac{d_h^{\pi^\star}(\tau_h)}{d_h^{\pi^{(t)}}(\tau_h)}\right)^2\right]}\varepsilon_{\mathsf{stat}}.$$

Therefore,

$$\text{Term (II)} \leq H\sqrt{\max_{t,h}\mathbb{E}_{\pi^{(t)}}\left[\left(\frac{d_h^{\pi^\star}(s_1,a_{1:h})}{d_h^{\pi^{(t)}}(s_1,a_{1:h})}\right)^2\right]}\varepsilon_{\mathsf{stat}}$$

Summing up the bounds of term (I) and (II) concludes our proof. □

Proof of Proposition 2. For each $\pi^{(t)}$, we have

$$\begin{aligned}
&J(\pi^\star) - \frac{1}{T}\sum_{t=1}^{T} J(\pi^{(t)}) \\
&= \frac{1}{T}\sum_{t=1}^{T}\sum_{h=1}^{H}\mathbb{E}_{\pi^\star}\left[A^{\pi^{(t)}}(s_h,a_h)\right] \\
&= \frac{1}{T}\sum_{t=1}^{T}\sum_{h=1}^{H}\mathbb{E}_{\pi^\star}\left[A^{\pi^{(t)}}(s_h,a_h) - A^{\pi^{(t)}}(\widehat{s}_h,a_h)\right] + \frac{1}{T}\sum_{t=1}^{T}\sum_{h=1}^{H}\mathbb{E}_{\pi^\star}\left[A^{\pi^{(t)}}(\widehat{s}_h,a_h)\right] \\
&= \underbrace{\frac{1}{T}\sum_{t=1}^{T}\sum_{h=1}^{H}\mathbb{E}_{\pi^\star}\left[\widehat{A}^{\pi^{(t)}}(\widehat{s}_h,a_h)\right]}_{\text{(I)}} + \underbrace{\frac{1}{T}\sum_{t=1}^{T}\sum_{h=1}^{H}\mathbb{E}_{\pi^\star}\left[A^{\pi^{(t)}}(\widehat{s}_h,a_h) - \widehat{A}^{\pi^{(t)}}(\widehat{s}_h,a_h)\right]}_{\text{(II)}} \\
&\quad + \underbrace{\frac{1}{T}\sum_{t=1}^{T}\sum_{h=1}^{H}\mathbb{E}_{\pi^\star}\left[A^{\pi^{(t)}}(s_h,a_h) - A^{\pi^{(t)}}(\widehat{s}_h,a_h)\right]}_{\text{(III)}},
\end{aligned}$$

where the first equation is a direct application of Lemma 3.

Term (I) can be bounded as $\leq H\varepsilon_{\mathsf{opt}}$ by Assumption 1.

For term (III), recall that $\Pr(s_h \neq \widehat{s}_h) \leq H\varepsilon_{\mathcal{P}}$, and therefore term (III) is bounded as:

$$\begin{aligned}
\mathbb{E}_{\pi^\star}\left[A^{\pi^{(t)}}(s_h,a_h) - A^{\pi^{(t)}}(\widehat{s}_h,a_h)\right] &\leq \Pr(s_h \neq \widehat{s}_h)\cdot\max_{s,\widehat{s},a}|A^{\pi^{(t)}}(s,a) - A^{\pi^{(t)}}(\widehat{s},a)| \\
&\leq (H\varepsilon_{\mathcal{P}})\cdot 2H = 2H^2\varepsilon_{\mathcal{P}}.
\end{aligned}$$

Next, for term (II), let $\Delta(s,a) = A^{\pi^{(t)}}(s,a) - \widehat{A}^{\pi^{(t)}}(s,a)$. This term can be bounded as:

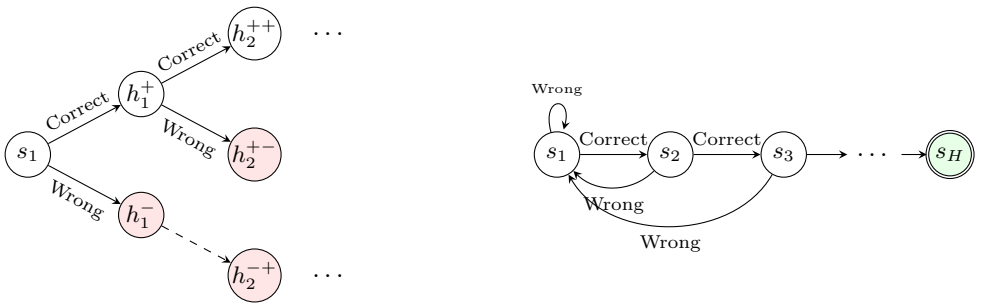
$$\begin{aligned}
&\mathbb{E}_{\pi^\star}\left[\Delta(\widehat{s}_h,a_h)\right] \\
=&\mathbb{E}_{\pi^\star}\left[\mathbb{E}[\Delta(\widehat{s}_h,a_h)\mid s_h,a_h]\right] \\
=&\sum_{s,a} d^{\pi^{(t)}}(s,a)\frac{d^{\pi^\star}(s,a)}{d^{\pi^{(t)}}(s,a)}\mathbb{E}[\Delta(\widehat{s}_h,a_h)\mid s,a] \\
\leq&\sqrt{\mathbb{E}_{\pi^{(t)}}\left[\left(\frac{d^{\pi^\star}(s_h,a_h)}{d^{\pi^{(t)}}(s_h,a_h)}\right)^2\right]}\sqrt{\mathbb{E}_{\pi^{(t)}}[(\mathbb{E}[\Delta(\widehat{s}_h,a_h)\mid s_h,a_h])^2]} && \text{(Cauchy-Schwarz)} \\
\leq&\sqrt{\mathbb{E}_{\pi^{(t)}}\left[\left(\frac{d^{\pi^\star}(s_h,a_h)}{d^{\pi^{(t)}}(s_h,a_h)}\right)^2\right]}\sqrt{\mathbb{E}_{\pi^{(t)}}[\Delta(\widehat{s}_h,a_h)^2]} && \text{(Jensen's inequality)}
\end{aligned}$$

$$= \sqrt{\mathbb{E}_{\pi^{(t)}}\left[\left(\frac{d^{\pi^\star}(s_h, a_h)}{d^{\pi^{(t)}}(s_h, a_h)}\right)^2\right]}\varepsilon_{\mathsf{stat}}.$$

Summing up the bounds of term (I), (II), and (III) concludes our proof.

□

B.4 Illustration of the Theoretical Analysis with Combination Lock



(a) Non-Markovian (History Tree)
State space grows as $\mathcal{O}(|\mathcal{A}|^H)$.
Exponential Coverage Needed.

(b) Markovian (State Chain)
State space grows as $\mathcal{O}(H)$.
Linear Coverage Sufficient.

Figure 6: Visual comparison of the state space complexity for the Combination Lock problem.

As shown in Figure 6, the state space complexity diverges significantly depending on how the agent represents its environment. In the action-sequence-based learning, the agent treats the entire history of actions as its state, which can be visualized as a *History Tree*. Each unique sequence of moves is treated as a distinct node in this tree. Consequently, the state space grows exponentially as $\mathcal{O}(|\mathcal{A}|^H)$. To learn an optimal policy in this regime, the agent requires exponential coverage of the history space, leading to the prohibitively high sample complexity.

In contrast, the Markovian formulation represents the environment as a *State Chain*. By mapping all action histories to a Markov state s_h, the agent collapses the complex tree structure into a linear progression of length H. This representation ensures that the state space grows only linearly, $\mathcal{O}(H)$, making linear coverage sufficient for convergence.

B.5 Proofs for Supporting Lemmas

Proof of Lemma 3. Let us prove by induction. When $H = 1$, the trajectory consists of a single step, and the statement is trivial.

Now assume that for horizon k the statement holds for any policy π' and π:

$$J_k(\pi') - J_k(\pi) = \sum_{h=1}^{k} \mathbb{E}_{(s_h,a_h)\sim d_h^{\pi'}}\left[A_h^{\pi}(s_h, a_h)\right].$$

For $H = k + 1$, we expand $J_{k+1}(\pi') - J_{k+1}(\pi)$:

$$\begin{aligned} J_{k+1}(\pi') - J_{k+1}(\pi) &= \mathbb{E}_{\pi'}\left[\sum_{h=1}^{k+1} r(s_h, a_h)\right] - \mathbb{E}_{\pi}\left[\sum_{h=1}^{k+1} r(s_h, a_h)\right] \\ &= \mathbb{E}_{\pi'}\left[\sum_{h=2}^{k+1} r(s_h, a_h)\right] - \mathbb{E}_{\pi'}[V_2^{\pi}(s_2)] \end{aligned} \tag{I}$$

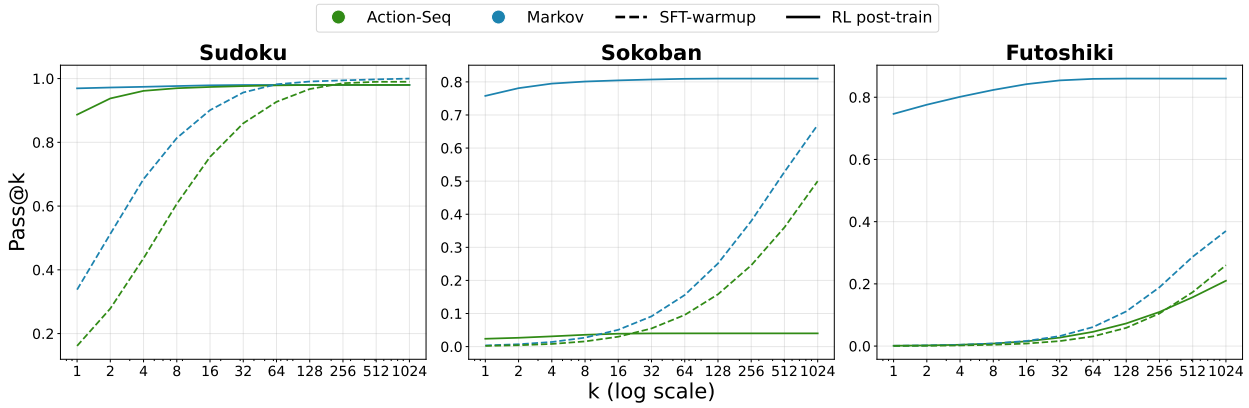


Figure 7: Pass@k accuracy for Qwen3-4B-based models. While action-sequence models rarely improve SFT Pass@k, Markov models consistently surpass their base models' limits.

$$+ \mathbb{E}_{\pi'}[r(s_1, a_1)] + \mathbb{E}_{\pi'}[V_2^{\pi}(s_2)] - \mathbb{E}_{\pi}\left[\sum_{h=1}^{k+1} r(s_h, a_h)\right] \tag{II}$$

Term I corresponds to the difference between expected returns of π' and π on an MDP with horizon k. Apply the inductive hypothesis to term I, we have

$$\mathbb{E}_{\pi'}\left[\sum_{h=2}^{k+1} r(s_h, a_h)\right] - \mathbb{E}'_{\pi}[V_2^{\pi}(s_2)] = \sum_{h=2}^{H} \mathbb{E}_{(s_h, a_h)\sim d_h^{\pi'}}\left[Q_h^{\pi}(s_h, a_h) - V_h^{\pi}(s_h)\right]$$

For term II, we have

$$\begin{aligned}
&\mathbb{E}_{\pi'}[r(s_1, a_1)] + \mathbb{E}_{\pi'}[V_2^{\pi}(s_2)] - \mathbb{E}_{\pi}\left[\sum_{h=1}^{k+1} r(s_h, a_h)\right] \\
&= \mathbb{E}_{\pi'}[Q_1^{\pi}(s_1, a_1)] - \mathbb{E}_{\pi}[V_1^{\pi}(s_1)] \\
&= \mathbb{E}_{s_1\sim d_1^{\pi'}}\left[\mathbb{E}_{a_1\sim\pi'(\cdot|s_1)}\left[Q_1^{\pi}(s_1, a_1)\right] - V_1^{\pi}(s_1)\right]
\end{aligned}$$

Summing up term I and II concludes our proof. □

C Additional Results

C.1 Pass@k Performance

We presents Qwen3-4B's Pass@k accuracy as k scales (Figure 7). Several distinct patterns emerge. For less challenging tasks like Sudoku, where SFT-warmup models already achieve high Pass@1024, both Markov and action-sequence models sharpen Pass@1 performance, with Markov models holding a slight advantage. However, on more difficult tasks like Sokoban and Futoshiki, action-sequence models fail to extend or even maintain the Pass@k of SFT models. In contrast, Markov models break through the capability boundaries of the base models, remarkably extending Pass@k.

C.2 Training Success Rate

We report the training-time success rate during RL post-training in Figure 8. For the experiments in Section 4.3, the corresponding training dynamics are shown in Figure 9. Across nearly all tasks and model variants, Markov models converge faster and achieve higher final success rates than action-sequence models. Introducing Markov states partially alleviates the slow-growth issue of action-sequence models in state–action-sequence variants, but a noticeable performance gap relative to Markov models remains.

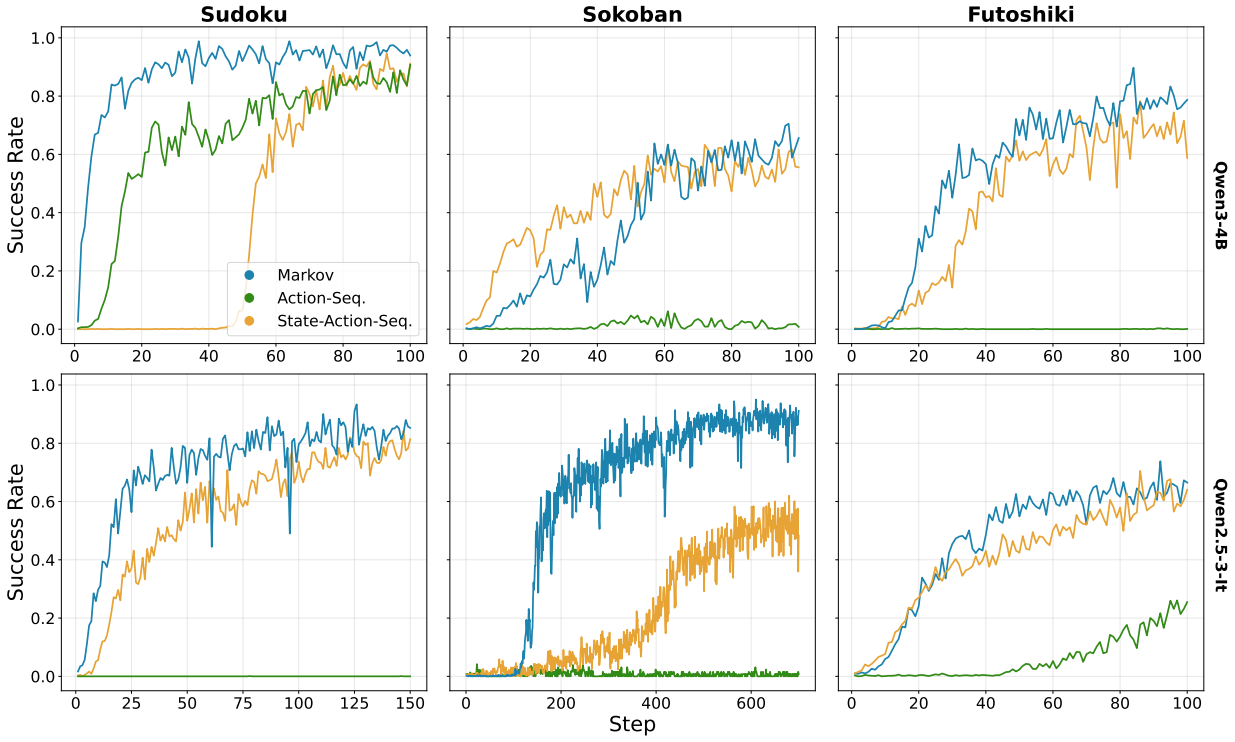


Figure 8: Training-time success rate during RL post-training.

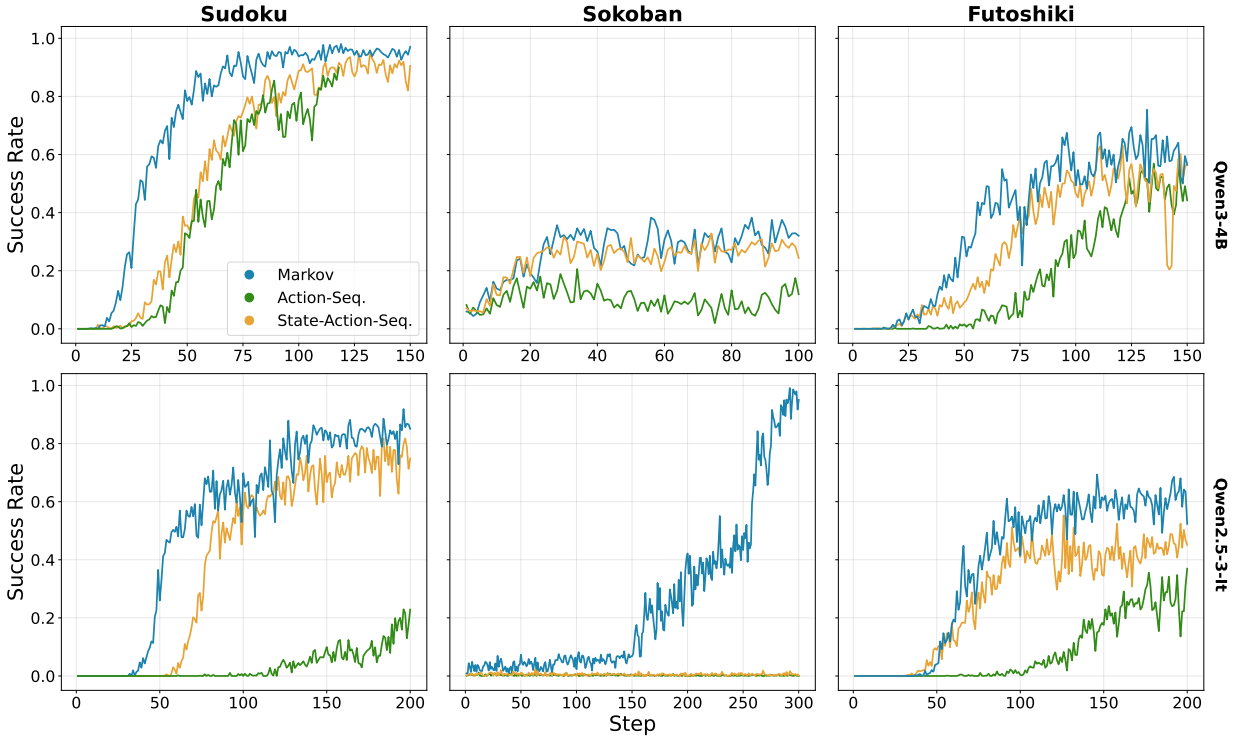


Figure 9: Training-time success rate during RL post-training for models with access to $A^\star$ in Section 4.3.

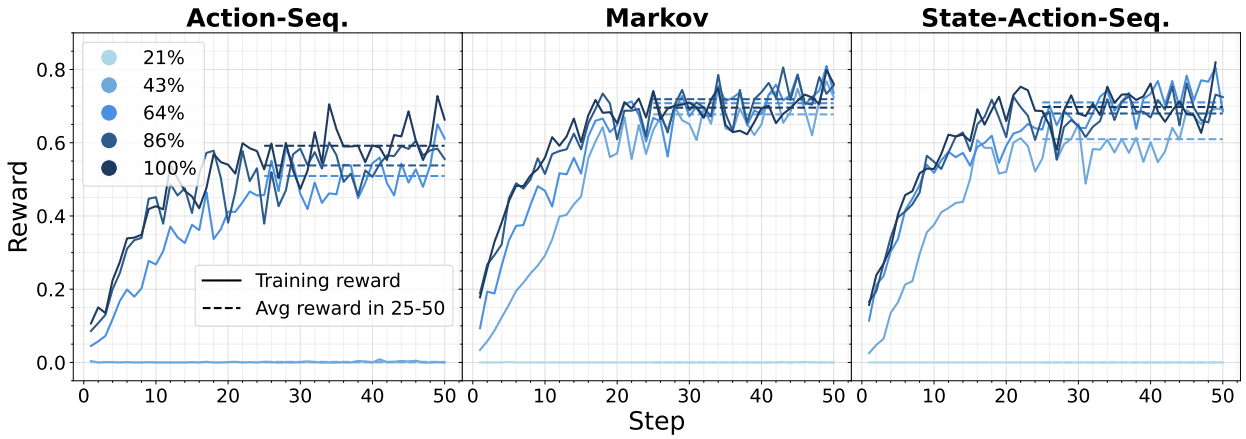


Figure 10: Ablation on the degree of SFT warm-up dataset. Experiments are conducted with Qwen2.5-3B-It on Sudoku.

C.3 Dependence on the Current State in State-Action-Sequence Models

Approach	Sudoku	Sokoban	Futoshiki
Qwen3-4B			
State-action-seq.	89.1	58.2	44.2
History-only	0.0	1.7	0.0
Last-state-only	39.2	3.1	26.6

Table 4: Analysis of the focus of the state-action-sequence models. **History-only**: the model is restricted to previous states and actions without access to the current state. **Last-state-only**: the model observes only the most recent state. All evaluations are conducted with the ground-truth state transition $\mathcal{P}^\star$.

To further probe what the state-action-sequence model actually uses after training, We test whether the model treats the current state as a sufficient statistic of the history. We evaluate two controlled variants: **(1)** History-only, which removes the current state s_h and provides only previous states and actions $\{(s_i, a_i)\}_{i=0}^{h-1}$, and **(2)** Last-state-only, which keeps only the most recent state s_h and discards the full history. Table 4 shows a stark pattern: History-only performance collapses to near zero across tasks, while Last-state-only retains a non-trivial fraction of the full state-action-sequence accuracy. Overall, these results indicate that well-trained state-action-sequence models rely primarily on the current state, with the action/state history contributing comparatively little, consistent with the view that they tend to internalize the Markov property of the state representation.

C.4 Ablation on the Degree of SFT Warm-up

To assess how the degree of SFT warm-up affects subsequent RL, we run an ablation over the fraction of SFT steps used to initialize the policy. We first train a base model with SFT to completion, saving intermediate checkpoints at 21%, 43%, 64%, 86%, and 100% of the total SFT steps. For each checkpoint, we then launch a fresh RL run, initializing the policy from that checkpoint while keeping all other RL settings fixed. As shown in Figure 10, 21% SFT is insufficient for all three approaches, with reward remaining at zero throughout RL. Continuing the warmup to 43%, the reward of Markov and State-Action-Seq starts ascending, whereas the Action-Seq. model still fails to escape the zero-reward plateau. Remarkably, even when fully SFTed (100%), the Action-Seq. model never reaches the final reward achieved by the 43%-SFT Markov model, highlighting that the Markov formulation is inherently easier for RL, more sample-efficient, and less reliant on heavy SFT. Comparing Markov and State-Action-Seq., we observe that with 43% SFT the latter learns more slowly and converges to slightly lower reward, but as the SFT fraction increases the State-Action-Seq. model gradually

internalizes the Markov structure of the environment and matches the performance of the Markov baseline.

C.5 Markov States in SFT

Approach	In Distribution						Out Of Distribution					
	Sudoku		Sokoban		Futoshiki		Sudoku		Sokoban		Futoshiki	
	Avg	Pass	Avg	Pass	Avg	Pass	Avg	Pass	Avg	Pass	Avg	Pass
Qwen3-4B												
Action-seq.	76.0	100.0	2.2	79.0	57.9	100.0	35.8	98.0	0.1	10.0	13.0	100.0
Markov	82.4	100.0	56.2	94.0	62.5	100.0	45.1	99.0	13.0	66.0	15.5	99.0
State-Action-seq.	70.4	100.0	53.6	98.0	61.0	100.0	33.1	99.0	14.9	58.0	14.8	100.0
Qwen2.5-3B-It												
Action-seq.	48.0	100.0	5.3	87.0	12.4	99.0	11.4	94.0	1.0	27.0	0.8	49.0
Markov	51.3	100.0	52.5	93.0	36.6	100.0	14.0	98.0	15.8	65.0	5.0	94.0
State-Action-seq.	62.0	100.0	35.7	95.0	50.3	100.0	19.9	97.0	5.5	58.0	9.4	98.0

Table 5: Performance comparison of different approaches for SFT.

We have demonstrated the importance of Markov states in RL post-training due to its lower sample complexity, it is interesting to investigate the case in supervised learning. Therefore, we SFT the models and show the evaluation results in Table 5. By comparing the approaches across models and tasks, we conclude two findings: **(1)** Action-sequence models consistently performs worse than Markov models and state-action-sequence models, most significantly on Sokoban, indicating the benefits of Markov states even in the supervised learning paradigm. By conditioning on an explicit (predicted) state provided by an external transition model, models with Markov states no longer need to reconstruct the current board configuration implicitly in its latent space, thereby offloading the burden of state tracking and prediction, therefore lower the sample complexity. **(2)** The gap between Markov models and state-action-sequence models shrinks or becomes unclear. This is because the SFT objective is to maximize likelihood from fixed offline trajectories rather than exploratory discovery. Unlike RL, which requires expansive coverage to reliably explore, SFT bypasses the need for the significant sample complexity reduction. Consequently, the Markov property is less critical in supervised settings.

C.6 Evaluation Results of Models with Access to $A^\star$

We present the full evaluation results on $\pi^{A^\star}_{\text{mkv}}$, $\pi^{A^\star}_{\text{act-seq}}$, and $\pi^{A^\star}_{\text{st-act-seq}}$ in Table 6 as a supplement to Section 4.3.

C.7 Examples of Solving Problems in Multiple Steps

Action-Seq. Model $\pi_{\text{act-seq}}$

State s_0

Here is the initial board state:

```
3 6 4 8 1 9 2 _ 7
5 9 8 3 7 2 6 1 4
_ 2 _ 5 6 4 9 3 8
8 1 5 4 3 6 7 9 2
9 _ 6 2 8 7 5 4 1
2 4 _ _ 5 1 3 8 6
4 8 2 6 9 3 1 7 5
1 5 3 7 2 8 4 6 9
```

Approach	In Distribution						Out Of Distribution					
	Sudoku		Sokoban[1]		Futoshiki		Sudoku		Sokoban[1]		Futoshiki	
	Avg	Pass	Avg	Pass	Avg	Pass	Avg	Pass	Avg	Pass	Avg	Pass
Qwen3-4B												
Action-seq. w/ A^*	90.8	93.0	18.2	23.0	54.8	71.0	61.0	78.0	2.7	8.0	11.7	21.0
Markov w/ A^*	97.8	100.0	33.0	33.0	64.8	73.0	80.9	85.0	4.0	4.0	25.2	35.0
State-Action-seq. w/ A^*	94.0	98.0	19.8	25.0	51.6	69.0	73.7	83.0	3.9	8.0	14.4	32.0
Qwen2.5-3B-It												
Action-seq. w/ A^*	28.9	44.0	0.2	5.0	44.1	63.0	2.9	9.0	0.0	0.0	3.8	16.0
Markov w/ A^*	83.4	86.0	94.2	97.0	54.2	59.0	48.8	57.0	82.5	87.0	26.3	36.0
State-Action-seq. w/ A^*	84.8	92.0	0.2	1.0	51.9	71.0	45.8	59.0	0.0	0.0	11.2	26.0

[1] Given the capability constraints of the base models, we use lower-complexity Sokoban tasks in this section.

Table 6: Performance comparison of $\pi^{A^*}_{\text{mkv}}$, $\pi^{A^*}_{\text{act-seq}}$, and $\pi^{A^*}_{\text{st-act-seq}}$ to show the difference of their generalization.

```
_ 7 9 1 4 5 _ 2 3

What is your immediate next move?
Action a0
\boxed{0,7,5}
State s1
Here is the initial board state:
3 6 4 8 1 9 2 _ 7
5 9 8 3 7 2 6 1 4
_ 2 _ 5 6 4 9 3 8
8 1 5 4 3 6 7 9 2
9 _ 6 2 8 7 5 4 1
2 4 _ _ 5 1 3 8 6
4 8 2 6 9 3 1 7 5
1 5 3 7 2 8 4 6 9
_ 7 9 1 4 5 _ 2 3

Here is the history of moves made so far:
Action taken: Fill cell at (0, 7) with 5
What is your immediate next move?
Action a1
\boxed{8,0,6}
State s2
Here is the initial board state:
3 6 4 8 1 9 2 _ 7
5 9 8 3 7 2 6 1 4
_ 2 _ 5 6 4 9 3 8
8 1 5 4 3 6 7 9 2
9 _ 6 2 8 7 5 4 1
2 4 _ _ 5 1 3 8 6
4 8 2 6 9 3 1 7 5
1 5 3 7 2 8 4 6 9
_ 7 9 1 4 5 _ 2 3
```

Here is the history of moves made so far:
Action taken: Fill cell at (0, 7) with 5
Action taken: Fill cell at (8, 0) with 6
What is your immediate next move?
...

Markov Model π_{mkv}

State s_0

Current board state:

```
3 6 4 8 1 9 2 _ 7
5 9 8 3 7 2 6 1 4
_ 2 _ 5 6 4 9 3 8
8 1 5 4 3 6 7 9 2
9 _ 6 2 8 7 5 4 1
2 4 _ _ 5 1 3 8 6
4 8 2 6 9 3 1 7 5
1 5 3 7 2 8 4 6 9
_ 7 9 1 4 5 _ 2 3
```

What is your immediate next move?

Action a_0

\boxed{0,7,5}

State s_1

Current board state:

```
3 6 4 8 1 9 2 5 7
5 9 8 3 7 2 6 1 4
_ 2 _ 5 6 4 9 3 8
8 1 5 4 3 6 7 9 2
9 _ 6 2 8 7 5 4 1
2 4 _ _ 5 1 3 8 6
4 8 2 6 9 3 1 7 5
1 5 3 7 2 8 4 6 9
_ 7 9 1 4 5 _ 2 3
```

What is your immediate next move?

Action a_1

\boxed{8,0,6}

...

State-Action-Seq. Model $\pi_{\text{st-act-seq}}$

State s_0

Here is the current board state:

```
3 6 4 8 1 9 2 _ 7
5 9 8 3 7 2 6 1 4
_ 2 _ 5 6 4 9 3 8
8 1 5 4 3 6 7 9 2
9 _ 6 2 8 7 5 4 1
2 4 _ _ 5 1 3 8 6
4 8 2 6 9 3 1 7 5
1 5 3 7 2 8 4 6 9
```

_ 7 9 1 4 5 _ 2 3

What is your immediate next move?

Action a_0

\boxed{0,7,5}

State s_1

Here is the history of moves made so far:

State of the board:

3 6 4 8 1 9 2 _ 7
5 9 8 3 7 2 6 1 4
_ 2 _ 5 6 4 9 3 8
8 1 5 4 3 6 7 9 2
9 _ 6 2 8 7 5 4 1
2 4 _ _ 5 1 3 8 6
4 8 2 6 9 3 1 7 5
1 5 3 7 2 8 4 6 9
_ 7 9 1 4 5 _ 2 3

Action taken: Fill cell at (0, 7) with 5

Here is the current board state:

3 6 4 8 1 9 2 5 7
5 9 8 3 7 2 6 1 4
_ 2 _ 5 6 4 9 3 8
8 1 5 4 3 6 7 9 2
9 _ 6 2 8 7 5 4 1
2 4 _ _ 5 1 3 8 6
4 8 2 6 9 3 1 7 5
1 5 3 7 2 8 4 6 9
_ 7 9 1 4 5 _ 2 3

What is your immediate next move?

Action a_1

\boxed{8,0,6}

State s_2

Here is the history of moves made so far:

State of the board:

3 6 4 8 1 9 2 _ 7
5 9 8 3 7 2 6 1 4
_ 2 _ 5 6 4 9 3 8
8 1 5 4 3 6 7 9 2
9 _ 6 2 8 7 5 4 1
2 4 _ _ 5 1 3 8 6
4 8 2 6 9 3 1 7 5
1 5 3 7 2 8 4 6 9
_ 7 9 1 4 5 _ 2 3

Action taken: Fill cell at (0, 7) with 5

State of the board:
3 6 4 8 1 9 2 5 7
5 9 8 3 7 2 6 1 4
_ 2 _ 5 6 4 9 3 8
8 1 5 4 3 6 7 9 2
9 _ 6 2 8 7 5 4 1
2 4 _ _ 5 1 3 8 6
4 8 2 6 9 3 1 7 5
1 5 3 7 2 8 4 6 9
_ 7 9 1 4 5 _ 2 3

Action taken: Fill cell at (8, 0) with 6

Here is the current board state:

3 6 4 8 1 9 2 5 7
5 9 8 3 7 2 6 1 4
_ 2 _ 5 6 4 9 3 8
8 1 5 4 3 6 7 9 2
9 _ 6 2 8 7 5 4 1
2 4 _ _ 5 1 3 8 6
4 8 2 6 9 3 1 7 5
1 5 3 7 2 8 4 6 9
6 7 9 1 4 5 _ 2 3

What is your immediate next move?

...

C.8 Thinking Mode Hinders Models from Solving Problems in Multiple Steps

As discussed in Section 4, we intentionally disable the model's reasoning mode and require it to output only the discrete action, without test-time chain-of-thought. In this section, we use the following example to show that models with enabled thinking mode fails to decompose the problem into independent, progressive decision steps.

Output of Qwen3-4B with thinking mode enabled

System prompt
You are playing Sudoku, a logic-based number placement puzzle.

Rules:
- Fill the grid with numbers from 1 to 9.
- Each number must appear exactly once in each row, column, and 3x3 subgrid.
- Empty cells are represented by underscores (_).

Please provide only your next move in the format: \boxed{row, col, number} where row and col are 0-indexed and number is between 1 and 9.

User prompt
Current board state:
3 6 4 8 1 9 2 _ 7

```
5 9 8 3 7 2 6 1 4
_ 2 _ 5 6 4 9 3 8
8 1 5 4 3 6 7 9 2
9 _ 6 2 8 7 5 4 1
2 4 _ _ 5 1 3 8 6
4 8 2 6 9 3 1 7 5
1 5 3 7 2 8 4 6 9
_ 7 9 1 4 5 _ 2 3
```

What is your immediate next move?

Please provide only your next move in the format: \boxed{row, col, number} where row and col are 0-indexed and number is between 1 and 9.

Model output

<think>
Okay, let's try to figure out the next move for this Sudoku puzzle. First, I need to look at the current board state and see where I can find a cell that has only one possible number that can fit. Let me start by going through each row, column, and subgrid to check for any missing numbers.
...
So existing numbers are 4,8,5,6,2,3,9. Missing are 1,7. Therefore, row 2 column 2 can be 1 or 7. Also, the subgrid (top-left) for row 2 column 2 is rows 0-2, columns 0-2. Existing numbers in that subgrid are 3,6,4 (row 0), 5,9,8 (row 1), and row 2: _,2,_. So existing numbers are 3,6,4,5,9,8,2. Missing are 1,7. Therefore, row 2 column 2 can be 1 or 7. So no unique move here. Therefore, not helpful.
So the possible moves are row 0 column 7 (5), row 8 column 0 (6), row 8 column 6 (8), row 5 column 2 (7). So which one is the next move? The problem is that the user may have a unique next move. But in the given puzzle, there might be multiple possibilities. However, the user wants the next move, so maybe the first one that is determined. Since row 0 column 7 is uniquely determined as 5, that's a valid move. Therefore, I think that is the correct answer.
</think>
To determine the **immediate next move** in the given Sudoku puzzle, we examine the current state of the board for any cell that can be uniquely filled based on the rules of Sudoku: each number from 1 to 9 must appear exactly once in every row, column, and 3x3 subgrid.

—

Analyzing the Cell at Row 0, Column 7
The current value in the row is:
“‘ 3 6 4 8 1 9 2 _ 7 “‘
This is **Row 0**, and the empty cell is at **Column 7**. Let's determine what number can go here. - The existing numbers in **Row 0** are: 3, 6, 4, 8, 1, 9, 2, 7. - The missing number in the row is **5**.
Now check if **5** is valid in **Column 7**: - Column 7 values are: 1, 3, 9, 4, 8, 7, 6, 2 (from rows 1 to 8). - So **5** is not in column 7 → it is **valid**.
Next, check the **subgrid** for **Row 0, Column 7** (the top-right 3x3 subgrid, rows 0–2, columns 7–8): - Existing values: 7 (from row 0, col 8), 1 (row 1, col 7), 4 (row 1, col 8), 3 (row 2, col 7), 8 (row 2, col 8). - No 5 is in the subgrid → valid.
Hence, **5 is the only number that can go into Row 0, Column 7**.

—

Final Answer

$$\boxed{0, 7, 5}$$

This example illustrates the model's behavior as an implicit transition model; it forecasts future board states within its reasoning trace and solves the problem holistically rather than through decomposition.

D Implementation Details

D.1 Datasets

As discussed in Section 4, we use three logical tasks during training and benchmarking. In this section, we provide details on these datasets for a more comprehensive setup description and reproducibility.

Dataset Sizes. As detailed in Section 4, we introduce a SFT warm-up phase before RL post-training. We use SFT dataset of different sizes to warm up different models on various tasks, due to the differences in base model capabilities and task complexities. The sizes are listed in Table 7. All models are trained for 1 epoch.

Model	Sudoku	Sokoban	Futoshiki
Qwen3-4B	4,000	4,000	4,000
Qwen2.5-3B-It	7,000	6,000	10,000

Table 7: SFT warm-up dataset sizes for each model on various tasks.

In RL post-training phase, we use a dataset of 10,000 samples for Sudoku, 6,000 samples for Sokoban, and 6,000 samples for Futoshiki. We additionally synthesize 100 problems as test set for each task. For experiments in Appendix C.5, we use larger SFT datasets than the one used in SFT warm-up, whose sizes are listed in Table 8. All models are trained with 1 epoch, except that Qwen2.5-3B-It is trained for 2 epochs on Sudoku dataset.

Model	Sudoku	Sokoban	Futoshiki
Qwen3-4B	18,000	20,000	24,000
Qwen2.5-3B-It	18,000×2	24,000	32,000

Table 8: SFT dataset sizes for each model on various tasks used in Appendix C.5.

In experiments conducted in Appendix C.4, we use the same dataset of size 18,000 training Qwen2.5-3B-It on Sudoku problems.

Dataset Difficulties. We synthesize the datasets using various configurations for different tasks. Details are presented in Table 9.

Benchmark	Sudoku	Sokoban	Futoshiki
Train set & ID Tests	Board size: 9×9 Number of blanks: 6	Board size: 9×9 Minimal number of steps: 6–10	Board size: 5×5 Number of blanks: 8–10
OOD Tests	Board size: 9×9 Number of blanks: 10	Board size: 9×9 Minimal number of steps: 12–14	Board size: 5×5 Number of blanks: 12–14

Table 9: Configurations of different tasks used in training and testing.

An exception is the Sokoban dataset used in Section 4.3. Given the capability constraints of the base models, we use lower-complexity Sokoban tasks. For in-distribution benchmarks, the minimal number of steps is between $2 \sim 4$; for out-of-distribution benchmarks, it is between $4 \sim 6$.

D.2 Training

We use rLLM (Tan et al., 2025) as the training and inference framework, which is backed by VERL (Sheng et al., 2024). We follow the most default hyperparameter during training. Specifically, we adopt a KL

divergence coefficient of 0.001, a learning rate of 1×10^{-6}, a batch size of 128, and sample 8 responses for each group. The mini-batch size is set to 128, except for training Qwen2.5-3B-It on Sokoban after SFT warm-up, where a size of 64 is used.

For SFT warm-up stage, we use VERL Sheng et al. (2024) to train the models. The batch size is set to 256 and the learning rate is chosen as 5×10^{-6}.

D.3 Training Details in Section 4.3

In Section 4.3, we use $A^\star$ to replace the estimated advantaged $\widehat{A}^{(i)}$ in GRPO. Particularly, the orginal objective of GRPO is

$$\mathcal{J}^{\mathsf{GRPO}}(\theta) = \mathbb{E}_{x\sim\rho,\{o^{(i)}\}_{i=1}^{G}\sim\pi_{\theta_{\mathsf{old}}}(\cdot|x)}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{h=1}^{|o_i|}\left\{\min\left[\frac{\pi_\theta(a_h^{(i)}\mid s_h^{(i)})}{\pi_{\theta_{\mathsf{old}}}(a_h^{(i)}\mid s_h^{(i)})}\widehat{A}^{(i)}, \mathrm{clip}\left(\frac{\pi_\theta(a_h^{(i)}\mid s_h^{(i)})}{\pi_{\theta_{\mathsf{old}}}(a_h^{(i)}\mid s_h^{(i)})}, 1-\epsilon, 1+\epsilon\right)\widehat{A}^{(i)}\right] - \beta D_{\mathrm{KL}}(\pi\|\pi_{\mathsf{ref}})\right\}\right], \quad (1)$$

where $\widehat{A}^{(i)} = \frac{r(x,o^{(i)})-\mathrm{mean}(\{r(x,o^{(1)}),\ldots,r(x,o^{(G)})\})}{\mathrm{std}(\{r(x,o^{(1)}),\ldots,r(x,o^{(G)})\})}$. Here, a_h represents the h-th token and s_h is the concatenation of all previous tokens.

Replacing $\widehat{A}^{(i)}$ with $A^\star$, the objective becomes

$$\mathcal{J}_{A^\star}^{\mathsf{GRPO}}(\theta) = \mathbb{E}_{x\sim\rho,\{o^{(i)}\}_{i=1}^{G}\sim\pi_{\theta_{\mathsf{old}}}(\cdot|x)}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{h=1}^{|o_i|}\left\{\min\left[\frac{\pi_\theta(a_h^{(i)}\mid s_h^{(i)})}{\pi_{\theta_{\mathsf{old}}}(a_h^{(i)}\mid s_h^{(i)})}A^{\star(i)}, \mathrm{clip}\left(\frac{\pi_\theta(a_h^{(i)}\mid s_h^{(i)})}{\pi_{\theta_{\mathsf{old}}}(a_h^{(i)}\mid s_h^{(i)})}, 1-\epsilon, 1+\epsilon\right)A^{\star(i)}\right] - \beta D_{\mathrm{KL}}(\pi\|\pi_{\mathsf{ref}})\right\}\right],$$

where the advantage of the optimal policy $A^{\star(i)}$ is calculated by the single-step response o_i.

Consider a response o. Let $\tilde{s}$ denote the current state (i.e., the board configuration for Markov models, and the history sequence for action-sequence and state–action-sequence models), and let $\tilde{a}$ denote the action represented by response o (i.e., the row index, column index, and value to fill for Sudoku and Futoshiki; or the movement direction for Sokoban). The computation of $A^\star(\tilde{s},\tilde{a})$ is task dependent.

For Sudoku and Futoshiki, we set the discount factor to $\gamma = 1$. For any valid board state $\tilde{s}$, the task is always solvable, and thus $V^\star(\tilde{s}) = 1$. If action $\tilde{a}$ transitions $\tilde{s}$ to another valid state $\tilde{s}'$—that is, the filled number does not violate any constraints and makes progress toward the final solution—then $Q^\star(\tilde{s},\tilde{a}) = 1$; otherwise, $Q^\star(\tilde{s},\tilde{a}) = 0$. Consequently, $A^\star(\tilde{s},\tilde{a}) = 0$ if action $\tilde{a}$ can still lead to a correct final solution, and -1 otherwise.

For Sokoban, we use a discount factor of $\gamma = 0.5$. Let n denote the minimum number of steps required to reach the goal from state $\tilde{s}$, and let n' denote the minimum number of steps after taking action $\tilde{a}$. The value n' may be infinite if $\tilde{a}$ leads to an unsolvable state. By definition, the optimal value and action-value functions are given by

$$V^\star(\tilde{s}) = \gamma^{n-1}, \qquad Q^\star(\tilde{s},\tilde{a}) = \gamma^{n'-1}.$$

Therefore,

$$A^\star(\tilde{s},\tilde{a}) = \gamma^{n'-1} - \gamma^{n-1}.$$

In practice, we compute n and n' using breadth-first search.

D.4 State Prediction Models

We train a state prediction model $\widehat{\mathcal{P}}$ based on Qwen2.5-3B-Instruct via SFT to predict the next state s_{h+1} from the current state s_h and action a_h. Particularly, we first collect triplets (s, a, s') from the environment $\mathcal{P}^\star$, where $s' = \mathcal{P}(s, a)$. We then use (s, a) as prompt and SFT the base model to predict s'. In practice, we use 174k samples to train a state prediction model for Sudoku, 91k samples for Sokoban, and 108k samples for Futoshiki. At test time, $\widehat{\mathcal{P}}$ replaces the environment $\mathcal{P}^\star$, enabling deployment without environment access.